

KATHOLISCHE UNIVERSITÄT
EICHSTÄTT-INGOLSTADT

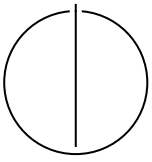
FACULTY OF MATHEMATICS AND GEOGRAPHY

CATHOLIC UNIVERSITY OF EICHSTÄTT-INGOLSTADT

Bachelor's Thesis in Data Science

Neural-Network-Based Lightweight Predecoder for Speculative Pauli Frame Updates

Andrei Dolmatov





KATHOLISCHE UNIVERSITÄT
EICHSTÄTT-INGOLSTADT

FACULTY OF MATHEMATICS AND GEOGRAPHY

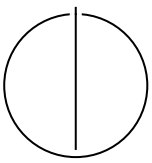
CATHOLIC UNIVERSITY OF EICHSTÄTT-INGOLSTADT

Bachelor's Thesis in Data Science

Neural-Network-Based Lightweight Predecoder for Speculative Pauli Frame Updates

Neuronales Netzwerk-basierter leichtgewichtiger Prädecoder für spekulative Pauli-Frame-Aktualisierungen

Author:	Andrei Dolmatov
Examiner:	Prof. Dr. Dominik Stöger, KUEI
Supervisor:	Aleksandra Świerkowska, TUM
Submission Date:	Submission date



Acknowledgments

I would like to thank the people that made this work possible. They helped me throughout the research, guided through the rough parts, and were always happy to help with any issue that might have occurred during the work.

My advisor at TUM, Aleksandra Świerkowska, who made this thesis possible in the first place, by helping me to arrange an external research as collaboration between Catholic University of Eichstätt-Ingolstadt and Technical University of Munich. She was always there to guide me and help with any kind of problem that was unfamiliar to me.

Am examiner from KUEI, Prof. Dr. Dominik Stöger, who happily supported my idea of an external research and helped me throughout the whole process.

And Prof. Dr. Pramod Bhatotia, for providing me with opportunity to work on this thesis at System's Research Group.

Abstract

Quantum error correction requires classical decoding that keeps up with the measurement stream. A lightweight predecoder could reduce the reference decoder's workload by handling records for which it can make a reliable prediction. We investigate a graph neural predecoder that estimates logical-observable flips from detector events and a graph of the error mechanisms supplied by a detector error model. A calibrated confidence gate decides which predictions to accept.

We evaluate recorded distance-five surface-code experiments from 53 Sycamore samples. The completed five-, nine-, and 25-round studies each compare twelve variants across two training schedules, two batch sizes, structured pruning, and mixed precision. Each study uses 397,500 held-out test shots. The highest neural accuracy is 62.32% at five rounds, 51.45% at nine rounds, and 50.08% at 25 rounds. At 25 rounds, all variants have AUCs close to 0.5. Stored correlated-matching predictions reach 82.11% and 73.19% on the same five- and nine-round test shots. Pruning reduces parameter count by 62.4%, and the mixed export reduces parameter-array storage by 80.8% relative to the baseline.

Median detector-to-prediction latency on an NVIDIA H200 NVL ranges from 4.39 to 4.83 ms. Mixed precision increases median latency in every tested condition. At the 1% accepted-error target, every fitted confidence gate delegates all shots. An exploratory threshold sweep measures the tradeoff between avoided reference calls and hybrid accuracy. At five rounds, the fixed-data, batch-64 baseline accepts 1.284% of shots at confidence threshold 0.985, introducing 18 additional errors and lowering overall accuracy by 0.00453 percentage points. The tested thresholds provide no evidence of useful workload reduction with preserved accuracy. The supplied classical-reference and threshold comparison covers five and nine rounds.

Contents

Acknowledgments ii

Abstract iii

1 Introduction 1

2 Background 3
- 2.1 Quantum computing . . . 3
- 2.2 Quantum error correction . . . 3
- 2.3 Quantum error-correcting codes . . . 4
  - 2.3.1 Repetition codes and stabilizers . . . 4
  - 2.3.2 Surface codes . . . 5
  - 2.3.3 Detection events . . . 6
- 2.4 Pauli frame . . . 7
- 2.5 Decoders . . . 7
  - 2.5.1 Circuit descriptions and detector error models . . . 7
  - 2.5.2 General definition of a decoder . . . 8
  - 2.5.3 Matching on a graph . . . 8
  - 2.5.4 Correlated matching . . . 8
- 2.6 Neural-network-based decoders . . . 9
  - 2.6.1 Graph neural networks . . . 9
  - 2.6.2 Model size and numerical precision . . . 10

3 Overview 11
- 3.1 Motivation . . . 11
- 3.2 Our proposal . . . 11
- 3.3 Scope of the investigation . . . 13

4 Design 14
- 4.1 Dataset . . . 14
- 4.2 Graph representation . . . 15
- 4.3 Neural-network predecoder architecture . . . 17

4.4 Training procedure . . . 18
4.5 Model compression . . . 19
4.6 Dynamic delegation system . . . 19

5 Implementation **21**
5.1 Software architecture and execution environment . . . 21
5.2 Data-loading pipeline . . . 22
5.3 Graph creation . . . 22
5.4 Neural-network-based predecoder implementation . . . 23
5.4.1 Graph convolutions . . . 23
5.4.2 Graph-level prediction . . . 24
5.4.3 Training and validation . . . 24
5.5 Training and experiment management . . . 25
5.6 Confidence-gated inference . . . 26
5.7 Model compression . . . 26
5.8 Evaluation and benchmarking . . . 27

6 Evaluation **28**
6.1 Methodology . . . 28
6.1.1 Data and partitions . . . 28
6.1.2 Training and model selection . . . 29
6.1.3 Metrics . . . 31
6.1.4 Classical reference and threshold comparison . . . 31
6.1.5 Hardware and timing procedure . . . 32
6.2 Prediction quality and training conditions . . . 33
6.2.1 Prediction results at five, nine, and 25 rounds . . . 33
6.2.2 Training exposure and duration . . . 35
6.3 Confidence gating and delegation . . . 38
6.3.1 Calibration and selection . . . 39
6.3.2 Gate outcomes . . . 39
6.3.3 Accuracy and reference-decoder workload . . . 40
6.4 Compression, latency, and memory . . . 43
6.4.1 Prediction quality and parameter storage . . . 44
6.4.2 Latency and throughput . . . 44
6.4.3 Runtime memory . . . 47
6.5 Comparison across round counts . . . 48

7 Related work **49**
7.1 Neural decoders 49
7.1.1 Early results 49
7.1.2 Graph neural networks 49
7.2 Neural predecoders 50

8 Summary and conclusion **51**

9 Future work **53**

Abbreviations **55**

List of Figures **56**

List of Tables **57**

Bibliography **59**

1 Introduction

Quantum computers rely on physical qubits whose states can change under the influence of noise. Quantum error correction addresses this problem by encoding a logical qubit across several physical qubits and repeatedly checking for errors. These checks produce measurement records that are then processed by a classical decoder.

Accurate decoding alone is not sufficient, as the classical system also needs to keep up with the measurement stream and provide correction information before an operation needs it. Recent surface-code experiments demonstrate that this is a practical concern: Google Quantum AI reports real-time decoding with a mean latency of 63 μs while the error-correction cycle lasts 1.1 μs [1]. These quantities describe different requirements. Throughput determines whether unfinished work accumulates, while latency determines how long a particular result takes to become available. Reducing the work required from the decoder could potentially help with both, given that the additional operations remain cheap.

In this thesis, we investigate a lightweight neural predecoder that runs before a reference (classical) decoder. It estimates whether a selected logical observable has flipped, using detector events and a graph of the possible error mechanisms. A dynamic delegation system accepts sufficiently reliable neural predictions and forwards the remaining records to the reference decoder. The purpose is to reduce the number of full decoding calls while preserving reliable logical predictions. This requires a balance between predictive quality, the fraction of accepted records, and the cost of neural inference. The latter is especially important, since we want the predecoder to be much faster than the reference decoder.

We develop the prototype for recorded surface-code experiments from the Sycamore dataset [2, 3]. We use a detector-error graph to retain the relationships supplied by each sample's detector error model, and train a graph neural network (GNN) to make one prediction per shot. We also investigate structured pruning and mixed precision, since a model that predicts accurately may still cost too much to use as a predecoder. The evaluation examines prediction quality, computational cost, and confidence gating on held-out data. We complete the five-, nine-, and 25-round studies using a shared training configuration. At five rounds, the strongest variant reaches

62.32% test accuracy, compared with 55.27% for the constant predictor. At nine rounds, all variants match the constant predictor's 51.45% accuracy. At 25 rounds, the strongest variant reaches 50.08% accuracy and AUC 0.5021, with predictions close to chance on the nearly balanced test set. Correlated matching reaches 82.11% and 73.19% on the same five- and nine-round test sets. Every fitted confidence gate delegates all shots at the 1% accepted-error target. A follow-up threshold sweep examines how accepting more neural predictions affects overall accuracy. At five rounds, higher confidence identifies subsets on which both decoders perform well, and the tested policies provide no evidence of useful workload reduction with preserved accuracy. These results make the quality of accepted predictions relative to the reference decoder central to further development.

Speculative Pauli-frame updates provide the broader application context. Our prototype produces logical-flip predictions and delegation decisions for complete experimental shots. Integrating it with a live quantum controller would require a Pauli-frame interface and measurements of the complete control path. The supplied classical-reference comparison covers five and nine rounds. The completed 25-round study adds neural prediction, gate, and timing results under the same experimental settings.

2 Background

2.1 Quantum computing

Quantum computing uses quantum states to represent and process information. Quantum algorithms rely on superposition, entanglement, and interference to perform certain computational tasks differently from classical algorithms. For a single qubit, the quantum state is given by

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \qquad |\alpha|^2 + |\beta|^2 = 1.$$

Here, $|0\rangle$ and $|1\rangle$ are the computational basis states, and α and β are complex probability amplitudes. Measurement in this basis returns 0 or 1 with probabilities $|\alpha|^2$ and $|\beta|^2$. The superposition does not give one direct access to both values at once, therefore an algorithm must use quantum operations to produce a useful measurement outcome [4].

Although quantum computers offer decreased computational times for some tasks, physical qubits remain sensitive to noise. Interaction with the environment can cause decoherence. Additionally, gates, initialization, and readout introduce further errors. These effects limit how long an unprotected quantum computation can maintain the relevant information.

2.2 Quantum error correction

Quantum error correction (QEC) addresses this issue by encoding logical information across several physical qubits. An encoded logical state is given by

$$|\psi_L\rangle = \alpha|0_L\rangle + \beta|1_L\rangle.$$

The logical basis states now describe states of several qubits. This encoding introduces redundancy that allows suitable measurements to gather errors without directly measuring the logical amplitudes.

A quantum error-correcting code defines the protected subspace and the checks that distinguish errors. These checks produce a syndrome, which a classical decoder uses to

infer correction information. However, this protection is limited, as different physical errors can produce the same syndrome, and some error combinations act on the logical information without producing a detectable error.

2.3 Quantum error-correcting codes

A quantum code is defined by $[[n, k, d]]$, where n is the number of physical data qubits, k is the number of logical qubits, and d is the code distance. The distance describes the smallest weight of an undetectable nontrivial logical Pauli operator. With ideal syndrome information, a distance-d code detects all Pauli errors of weight at most $d-1$ and corrects arbitrary errors on at most

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor$$

qubits. The encoding rate $R = k/n$ describes how much logical information the data qubits store [5].

2.3.1 Repetition codes and stabilizers

A simple example is the three-qubit repetition logical encoding

$$\alpha|0\rangle + \beta|1\rangle \longmapsto \alpha|000\rangle + \beta|111\rangle.$$

Three physical qubits occupy an eight-dimensional Hilbert space, but the encoded states lie in the two-dimensional code space

$$\mathcal{C} = \mathrm{span}\{|000\rangle, |111\rangle\}.$$

This encoding protects against a single bit flip. Its bit-flip distance is 3, but its distance against arbitrary quantum errors is 1, meaning that a single phase flip can already change the logical state.

The Pauli operators

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

demonstrate these two error types. The X operator exchanges the basis states, while Z changes their relative phase. They anticommute, $XZ = -ZX$, which lets parity checks distinguish errors that anticommute with the corresponding check.

A stabilizer S leaves every state in the code space unchanged:

$$S|\psi_L\rangle = |\psi_L\rangle.$$

For the repetition code one chooses the commuting checks

$$S_1 = Z_1Z_2, \qquad S_2 = Z_2Z_3.$$

Their measurement outcomes are eigenvalues $+1$ or -1, which we encode as bits 0 and 1. Valid code states give $+1$ for both checks. Suppose a bit flip affects the second qubit. The state then becomes

$$X_2|\psi_L\rangle = \alpha|010\rangle + \beta|101\rangle.$$

Both checks now give -1. The syndrome identifies the middle-qubit error under the assumption that at most one bit flip occurred. It does not reveal α or β.

We measure such parity information using ancilla qubits. For example, two CNOT operations from data qubits to an ancilla initially in $|0\rangle$ act on basis states as

$$|b_1b_2\rangle|0\rangle_a \longmapsto |b_1b_2\rangle|b_1 \oplus b_2\rangle_a.$$

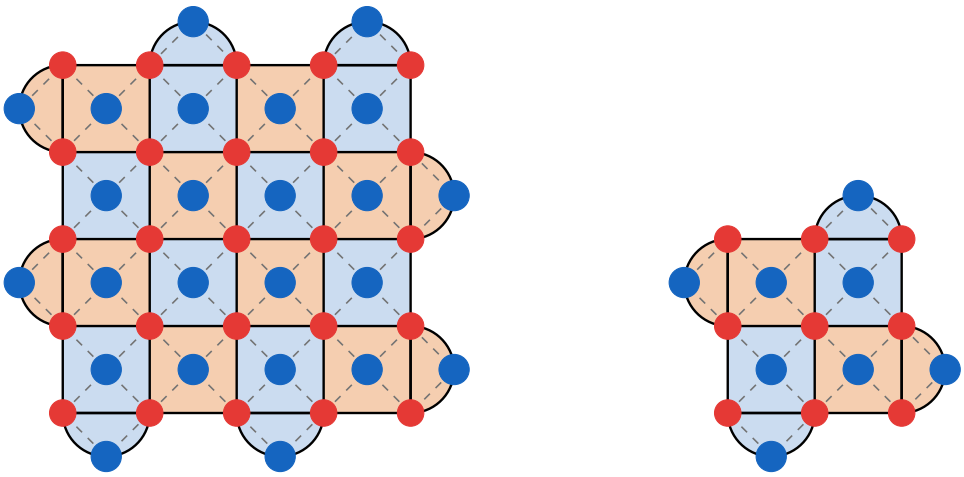
Measuring the ancilla reveals the Z_1Z_2 parity without distinguishing $|00\rangle$ from $|11\rangle$. Appropriate basis changes allow analogous measurements of other Pauli checks.

2.3.2 Surface codes

The repetition code encoding illustrates error detection but does not protect against both bit and phase errors. Surface codes arrange local checks on a two-dimensional lattice so that they can protect both components of the logical state. Their local connectivity is useful for hardware in which qubits interact primarily with neighboring qubits [2].

Figure 2.1 shows the stabilizer supports of two rotated surface-code patches. A distance-five patch uses 25 data qubits and 24 measurement ancilla qubits, while a distance-three patch uses 9 data qubits and 8 ancilla qubits. Bulk checks involve four data qubits and boundary checks involve two.

One round of syndrome extraction gives information about the checks at one point in time. Repeating the rounds adds a temporal dimension to the decoding problem, incorporating error evolution that accumulates over the course of the experiment. The resulting space-time graph describes full temporal measurement history.



(a) Distance-five patch.

(b) Distance-three patch.

Figure 2.1: Schematic stabilizer supports of the surface-code patches. Red circles denote data qubits and blue circles denote ancilla qubits. The distance-five patch contains 25 data qubits and 24 ancilla qubits, the distance-three patch contains 9 data qubits and 8 ancilla qubits. Dashed lines indicate stabilizer support connections.

2.3.3 Detection events

Let $s_i^{(t)}$ denote the binary outcome of check i in round t. For repeated checks with a constant expected parity, we define an interior detection event by

$$d_i^{(t)} = s_i^{(t)} \oplus s_i^{(t-1)}, \qquad t \in \{2, \ldots, r\}.$$

In a noiseless experiment, consecutive outcomes agree and the detector returns zero. A change produces a detection event. The first and last rounds require comparisons with the preparation and final measurements. The circuit description specifies these boundary detectors.

Consider an isolated readout error in an otherwise constant check history:

$$\ldots, 0, 0, 1, 0, 0, \ldots.$$

The incorrect outcome differs from both its predecessor and successor, so it produces two consecutive detection events. A data error that changes a check parity and persists can instead produce

$$\ldots, 0, 0, 1, 1, 1, \ldots.$$

This history contains one change at the first error occurrence, followed by new persisting outcomes. Other checks and the temporal boundaries supply additional information. A decoder uses these spatial and temporal relationships to distinguish plausible explanations of the observed events.

2.4 Pauli frame

A decoder does not always need to trigger a physical correction gate. The classical controller can instead track a pending Pauli correction in a Pauli frame and account for it in subsequent operations and measurements. This avoids adding a physical gate whenever the correction can remain a classical record. This is useful since gates themselves can add additional noise, which will further affect the signal quality.

For a Clifford operation U, conjugation maps a Pauli operator F to another Pauli operator, $UFU^{\dagger}$. The controller can therefore update the frame through Clifford operations without leaving the Pauli representation [4]. However, later measurements or operations that depend on the correction still require the relevant information in time.

2.5 Decoders

2.5.1 Circuit descriptions and detector error models

A decoder needs a description of the detection event effects that an error might produce. Stim provides a text format for stabilizer circuits and detector error models [6]. A circuit specifies operations, measurement records, detector parities, and logical observables. Coordinate annotations additionally describe detector geometry, according to the convention used by the data.

A detector error model (DEM) describes possible error mechanisms through their probabilities and their effects on detectors and logical observables. For instance,

```
error(0.01) D3 D7 L0
```

describes a mechanism with probability 0.01 that flips detectors 3 and 7 and logical observable 0. The record describes only a possibility, not guaranteeing that this mechanism occurred in a particular shot. Multiple mechanisms are combined by the parity - two flips of the same target cancel.

2.5.2 General definition of a decoder

A QEC decoder uses the syndrome and the code structure to infer a correction. Let $\mathbf{d} \in \{0,1\}^{n_D}$ contain the detector events and let H be the detector-error incidence matrix. An entry H_{ij} is 1 when mechanism j flips detector i. If $\mathbf{e}$ represents the occurring mechanisms in this model, then

$$H\mathbf{e} = \mathbf{d} \pmod 2.$$

The decoder looks for an explanation $\hat{\mathbf{e}}$ of the observations, or directly predicts the correction relevant to a logical observable. Successful recovery does not require recovering the exact physical error, as combined error and correction must act trivially on the logical state.

2.5.3 Matching on a graph

A matching on a graph is a set of edges that share no vertices. A perfect matching covers every vertex, and a minimum-weight perfect matching (MWPM) has the smallest total edge weight among such matchings.

In surface-code decoding, a graphlike error mechanism connects two detectors, or a detector to a boundary. Its weight reflects its probability. For an observed syndrome, the decoder finds low-cost paths that pair the triggered detectors with one another or with suitable boundaries. The selected paths supply an estimated correction. Thus, the matching problem uses distances derived from the decoding graph, rather than simply treating every pair of triggered detectors as one physical error.

An ordinary matching graph cannot directly represent a single mechanism with more than two detector targets. Decomposing that mechanism into edges makes matching possible, but treating the resulting edges independently loses their common origin. Correlated matching addresses part of this limitation.

2.5.4 Correlated matching

Correlated matching uses information about error mechanisms that affect several graph edges together [7]. Consider the DEM record

```
error(0.001) D1 D2 ^ D3 D4
```

The separator suggests two graphlike components of the same mechanism. It does not turn them into independent errors. Selecting the edge between D_1 and D_2 therefore provides information about the edge between D_3 and D_4.

A first matching pass supplies an initial hypothesis. A correlation-aware pass can then adjust edge weights according to that hypothesis and repeat the matching. The resulting correction can differ from the first one. This use of correlations can improve decoding, although a particular reweighting procedure does not guarantee the optimal correction for every record.

2.6 Neural-network-based decoders

Neural networks provide another way to infer correction information from a syndrome. Early work includes a restricted Boltzmann machine that learns error and syndrome statistics for a topological code [8]. Other models predict a logical error class directly. These approaches share the decoding task but do not necessarily produce the same intermediate outputs.

Let L be the flip indicator for a binary logical observable. A probabilistic neural decoder with trainable parameters θ can learn

$$p_\theta(\mathbf{d}) \approx P(L = 1 \mid \mathbf{d}).$$

A corresponding decision is

$$\widehat{L} = \mathbf{1}[p_\theta(\mathbf{d}) > 0.5].$$

A direct logical classifier does not need to reconstruct the physical error vector. Its probability estimate also provides a possible basis for accepting or rejecting a prediction. However, a large output confidence alone does not establish a small error probability.

2.6.1 Graph neural networks

Graph neural networks represent observations as node features together with a connectivity structure. A message-passing layer combines information from neighboring nodes and applies learned transformations. Repeating these layers allows information to travel over several graph edges. A readout operation then aggregates node representations into a graph-level prediction. This structure is useful when the relationships between measurements carry information that a flat input vector does not express explicitly. For instance, Lange et al. apply graph neural networks to quantum decoding using both simulated and experimental data [9].

2.6.2 Model size and numerical precision

The size of a neural network affects parameter storage, intermediate activations, and computational workload. Structured pruning removes complete channels or units, producing smaller tensor dimensions. Unstructured pruning on the other hand assumes setting some of the weights to zero, which does not necessarily reduce the computational workload, unless the resulting sparsity is exploited appropriately. Mixed precision uses different numerical formats for different operations. For instance, lower-precision matrix operations can coexist with single-precision accumulation or outputs. This can reduce some costs while retaining higher precision where numerical stability matters [10]. The actual benefit depends on the hardware and operations involved. Neither a smaller parameter count nor a narrower numerical format alone establishes a reduction in application-level latency.

3 Overview

3.1 Motivation

Quantum error correction requires a decoder that is both accurate and fast. If the decoder processes syndrome information slower than the QPU produces it, unfinished computations accumulate. Qubits continue to experience noise while the system works, so waiting for the decoder might potentially introduce additional noise. Experiments with real-time surface-code decoding make this constraint relevant beyond simulated workloads [1].

A predecoder could reduce this workload by handling records that do not require the full reference decoder, meaning records that are fairly easy to decode. However, since predecoder is the very first decoding component that is supplied with the syndrome, it has to go through all of the records, even the ones that will end up discarded and delegated to the reference decoder. The benefit of a neural predecoder therefore depends on more than classification accuracy. We need to consider how many predictions we can accept, how reliable they are, and whether the saved decoding work compensates for the predecoder's own execution time and memory requirements.

This thesis investigates the following research question:

> *How can a lightweight neural predecoder reduce reference-decoder workload while maintaining reliable logical-flip predictions and limiting the additional execution time and memory use?*

3.2 Our proposal

We propose a combination of a neural predecoder, a dynamic delegation system, and a reference decoder. For each experimental shot, the predecoder receives a detection event vector derived from the measurement record. We place these events on a graph that connects detectors to the error mechanisms that can affect them. The network uses this representation to estimate the probability that the selected logical observable has flipped.

The delegation system decides whether to accept the neural prediction. Accepted records bypass the reference decoder, leaving an appropriate correction operation in the corresponding Pauli frame component, while rejected records follow the reference path. Both paths return the same type of output: a logical-observable-flip prediction. We fit the gate using calibration records that remain separate from network training, and fix its acceptance rule before evaluating test shots. This lets us examine the reliability of the accepted predictions without choosing the rule from their test outcomes.

Figure 3.1 describes this proposal in the context of a quantum controller at a high level. The neural path could supply an early update to the relevant Pauli-frame component, while the full decoder handles records that the gate rejects. We do not run a second decoder to verify every accepted estimate or implement a rollback mechanism.

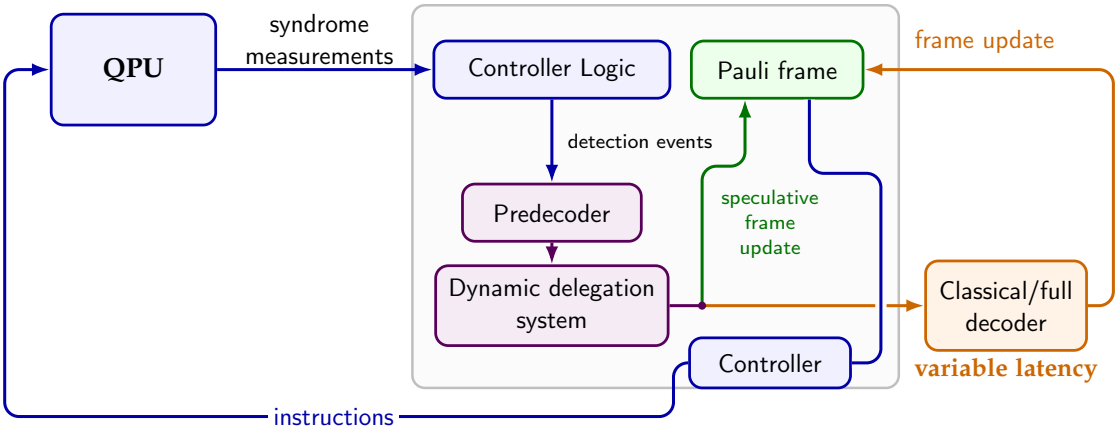


Figure 3.1: Conceptual quantum error-correction pipeline with speculative predecoding. The implemented prototype covers neural prediction, confidence gating, and inference benchmarking. Stored classical-decoder predictions support the offline quality comparison. The QPU, controller, and live reference path illustrate the intended application context.

In the recorded memory experiments, the predicted flip corrects the interpretation of the measured logical outcome. This task does not require identifying the exact physical error that occurred. It also does not determine every component of a Pauli frame. We use this limited task to study whether confidence-based delegation is useful before extending the approach to a controller interface.

We keep the neural model small and investigate structured pruning and mixed-precision inference to reduce its cost further. These changes may also affect prediction confidence, so we evaluate each eligible model with its own fitted gate.

3.3 Scope of the investigation

We use recorded surface-code experiments from the Sycamore dataset supplied with circuit descriptions, detector error models, and logical-flip labels [2, 3]. These records let us study the real experimental noise represented in the collection. We select the distance-five, X-basis configuration from all 53 samples. The completed studies cover five, nine, and 25 rounds, with test and timing results for all twelve variants at each round count. We reserve separate shots for training, validation, calibration, and testing. We compare two ways of consuming the training records: repeated passes over a fixed pool and successive portions of previously unused shots. We also compare training batch sizes and compressed versions of the model. Within each round-count study, all conditions share the held-out shot identifiers. We distinguish predictive performance from deployment performance. The first concerns logical-flip prediction and the reliability of the gate. The second concerns execution time and resource consumption on the selected classical hardware. Our timing measurements process complete-shot records in an offline software pipeline. Sub-microsecond inference on actual hardware remains a possible deployment target, whose feasibility requires measurement.

A follow-up analysis combines saved neural predictions with the dataset's correlated-matching predictions at five and nine rounds. It compares fixed confidence thresholds through their accepted coverage and their effect on hybrid accuracy. We retain the original fitted gates as a separate evaluation and treat the threshold sweep as exploratory.

4 Design

In this chapter we explain the choices behind the predecoder introduced in Chapter 3. We start with the available experimental data, then describe how we represent it as a graph and use it to train the network. We also explain how compression and confidence gating support the aim of reducing reference-decoder computational workload. The design accounts only for one logical-observable per complete shot, as it is presented in the data.

4.1 Dataset

We use the surface-code collection associated with the work on decoder priors by Sivak et al. [3], containing experimental records from Google's Sycamore surface-code experiments [2]. We choose these recordings because they expose the model to real experimental noise and come with the circuit and error-model information needed to interpret the detector records. We do not generate new synthetic training labels.

The dataset contains 53 samples. Each sample contains a distance-five configuration and 5 distance-three configurations at different patch locations. The distance-five patch uses 25 data qubits and 24 ancilla qubits, a distance-three patch uses 9 data qubits and 8 ancilla qubits, as illustrated in Figure 2.1. The available recordings cover $r \in \{5, 9, 13, 17, 21, 25\}$ rounds.

For the main study, we select the distance-five, X-basis configuration from every sample. We complete the neural comparisons at five, nine, and 25 rounds. Within each round count, we keep the circuit configuration fixed while comparing training and compression choices. Each sample supplies its own circuit, DEM, and experimental records.

4 file inputs are sufficient for the proposed pipeline. The ideal (noiseless) Stim circuit defines the detectors and their coordinate annotations. The correlation-derived DEM describes candidate mechanisms and their effects. The detector-event records provide the observations for individual shots, and the supplied logical-observable-flip records provide the (ground-truth) binary labels. For a memory experiment, the label indicates

whether the measured logical outcome differs from its expected value. We use this supplied label directly rather than inferring the occurrence of every physical mechanism.

The circuit and DEM have different purposes. The ideal circuit describes the noiseless (idealized) circuit used in the quantum experiment, while DEM supplies the error-related information. We keep the supplied DEMs fixed throughout the model comparison and account for this choice when describing the limits of the separate evaluation set.

4.2 Graph representation

We represent each sample by a bipartite detector-error graph

$$G = (V_D \sqcup V_E, \mathcal{E}), \qquad (D_i, E_j) \in \mathcal{E} \iff D_i \in \mathcal{D}_j.$$

Here, V_D - detector nodes, V_E contains one node for each DEM error instruction, and $\mathcal{D}_j$ is the detector support of error instruction j. An error node also carries the mechanism probability p_j and information about its logical support $\mathcal{L}_j$.

We choose this representation because a single mechanism can affect more than two detectors. Connecting all of its targets through one error node retains their common origin without replacing the mechanism by independent pairwise errors. For instance, a mechanism involving four detectors has four incidences to the same error node. Decomposition separators remain part of the original mechanism description. We combine repeated targets by parity, since two flips of a detector or observable cancel out, giving the original value.

Figure 4.1 shows how the circuit and DEM contribute to the graph. The circuit supplies spatial positioning of data and ancilla qubits, as well as temporal information, while the DEM supplies the error mechanisms with their respective probabilities.

The resulting graph is built from static and dynamic part. Connectivity, coordinates, and mechanism priors remain fixed within one sample. Only the detector-event input changes from shot to shot. This separation lets us reuse the same structure and avoids rebuilding a graph for every observation.

We use 13 node-feature channels. They distinguish detector and error nodes and describe position, time, error mechanism probability, and boundary or logical-support information. One channel holds the detector's observed event bit. Its value remains zero on error nodes. Coordinate and time normalization keep the geometric values on comparable scales.

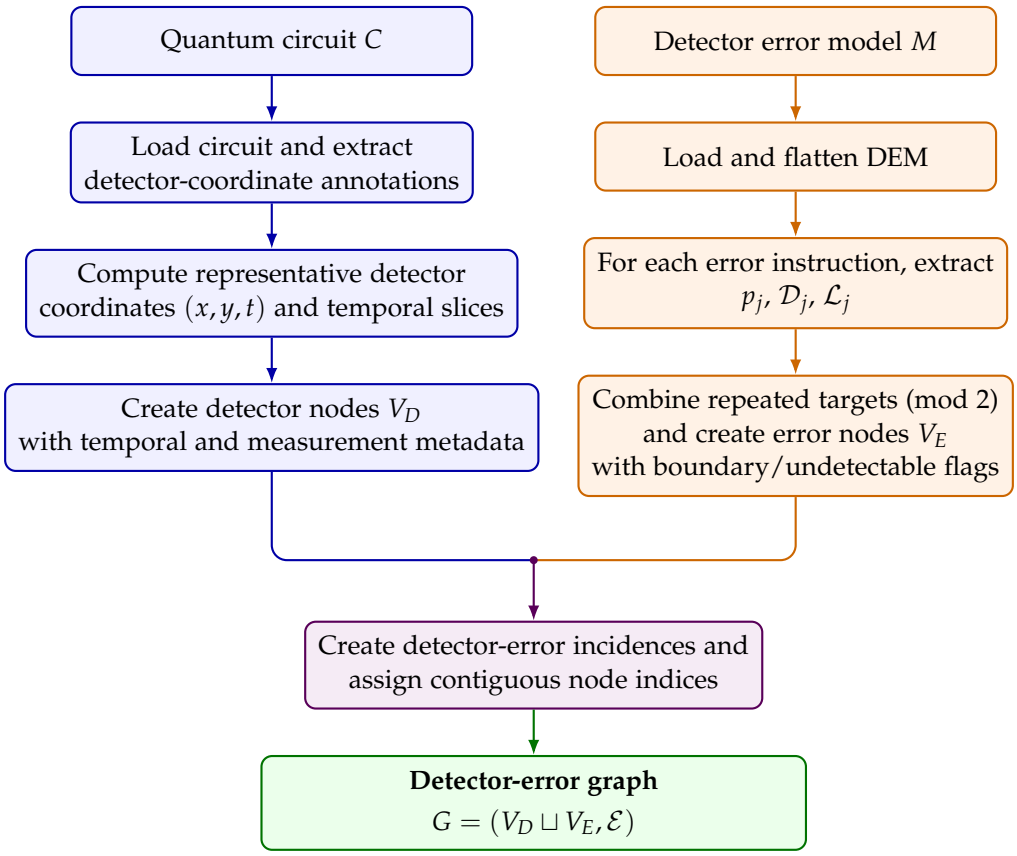


Figure 4.1: Construction of the detector-error graph from the quantum circuit and detector error model. The circuit branch provides detector geometry and temporal information, while the detector error model provides candidate error mechanisms with their probabilities p_j, detector support $\mathcal{D}_j$, and logical support $\mathcal{L}_j$, where j is the error instruction index.

We add self-loops to allow the retention of node's own information during aggregation. For the undirected incidence adjacency A, we use

$$\widehat{A} = \widetilde{D}^{-1/2}(A + I)\widetilde{D}^{-1/2}, \qquad \widetilde{D}_{ii} = \sum_j (A + I)_{ij}.$$

This normalization limits the effect of differences in node degree. Its coefficients describe aggregation, while the DEM probabilities remain node features. Keeping these quantities separate lets the network use error likelihoods without treating them as graph-normalization weights.

4.3 Neural-network predecoder architecture

We use a graph neural network because the detector-error relationships provide a natural neighborhood for exchanging information. The same learned transformations act on every node, so the model can process the sample-specific graphs without allocating separate parameters for each detector. Our baseline projects the 13 input features to 96 hidden channels and applies 3 residual graph-convolution blocks.

A graph convolution combines transformed features according to $\widehat{A}$. Schematically, a block computes

$$Z^{(k+1)} = \mathrm{ReLU}\big(\widehat{A}H^{(k)}W^{(k)} + b^{(k)}\big),$$

then adds the previous representation $H^{(k)}$, applies layer normalization, and uses dropout during training. The hidden width remains constant so that the residual addition preserves each node's feature dimensions. We use residual connections to retain the previous representation while learning additional neighborhood information. Dropout with rate 0.1 provides regularization when training revisits the same recordings.

In the bipartite graph, one step connects detectors and mechanisms, while two steps allow detectors with a common mechanism to exchange information. Three blocks provide a decent message-passing depth without making the network large, however this design remains a baseline choice.

The readout combines mean, maximum, and attention-weighted pooling over the nodes. Mean pooling summarizes the graph, maximum pooling retains strong feature responses, and attention lets the model weight node contributions to the final prediction. We concatenate these summaries and use a fully connected head with 128 and 64 units. A sigmoid output produces one logical-flip probability.

We choose a graph-level prediction because the task concerns one selected logical observable, rather than a correction at every physical qubit. This keeps the output small, however it does not remove the cost of processing the graph, since the intermediate edge messages still grow with batch size, hidden width, and the number of incidences. These costs additionally motivate the compression and runtime comparisons.

4.4 Training procedure

We train the network as a binary classifier using the recorded logical-flip labels. For a batch of B shots, with predicted probabilities p_i and labels ℓ_i, the loss is binary cross-entropy,

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\big[\ell_i \log p_i + (1-\ell_i)\log(1-p_i)\big].$$

We clip probabilities before taking logarithms to avoid numerical instability. We choose a probability-based loss because the confidence gate needs a more informative resulting model value, for which a hard binary decision is not suitable. We use the Adam optimizer to update the trainable parameters [11].

We separate training, validation, calibration, and test shots within each sample. Training updates the network, validation selects its weights and checks compressed candidates, calibration fits the gate, and testing assesses the fixed system. This separation prevents gate selection from using the same outcomes that later measure its performance. All conditions reuse the same partition identifiers for a direct comparison.

We compare two training schedules. In fixed-dataset mode, every epoch visits the same selected training subset in a new order. In fresh-data mode, each epoch consumes up to a configured number of unused shots from a shuffled pool. The last portion may be smaller, and training stops when no unused records remain. Fresh data in fresh-mode means previously unseen samples data from the dataset.

These schedules let us examine the effect of repeated exposure to a finite dataset. We record unique shots, total training presentations, completed epochs, and elapsed time to describe their training budgets. A batch contains several records from one sample and uses its graph, while the model shares weights across the collection.

Validation-loss early stopping limits training when further epochs no longer produce a sufficient improvement. The baseline has an upper limit of 100 epochs. We keep one initialization seed and compare batch sizes 64 and 128 to fit the available experimental budget. These choices provide controlled conditions, but do not estimate variability

across initializations. Chapter 6 gives the partition fractions, stopping criteria, and complete benchmark configuration.

4.5 Model compression

We investigate structured pruning because removing whole channels produces smaller tensor operations without requiring a sparse-weight execution engine. The pruned candidate reduces the hidden width from 96 to 64 and the prediction head from 128/64 to 64/32 units. It retains all three graph blocks and the same detector-error input, so the comparison changes network capacity while preserving the information available to it.

We rank channels using weight magnitudes and transfer the retained weights to a smaller network. The residual blocks require a consistent selection of hidden channels across their connected layers. Fine-tuning then allows the smaller model to adapt to the removed channels. We limit this stage to 20 epochs with validation-loss early stopping. For both initial training schedules, fine-tuning revisits a fixed subset of the training pool. A compressed model's training history includes this repeated exposure and the initial training of its parent.

Next, we create a mixed-precision version of an eligible pruned model. Lower-precision feature transformations can reduce arithmetic and storage costs, while graph aggregation, normalization, pooling, and the output retain single precision. This choice protects reductions and the probability estimate used by the gate. The conversion requires no extra training, and it is distinct from integer quantization.

We check each compressed candidate on validation data before admitting it to the test comparison. A small validation-loss increase is acceptable under the predefined criterion, but it does not establish unchanged test error or confidence. Each eligible variant therefore receives its own calibrated gate. We measure storage, memory, and execution time separately, since a smaller network does not have to improve all three.

4.6 Dynamic delegation system

The delegation system decides which neural predictions can bypass the reference decoder. We use the probability output instead of a fixed rule based only on detector count, since two equally sparse records can have different logical implications. The gate should respond to the model's uncertainty about the prediction, rather than assume that few events always imply an easy decoding case.

Let q be the predicted probability after temperature scaling. We define confidence as

$$c = \max(q, 1 - q), \qquad \text{accept if } c \geq \tau.$$

This symmetric rule treats a confident no-flip prediction and a confident flip prediction in the same way. Temperature scaling adjusts confidence without changing the binary decision for a positive temperature. We fit it on one part of the calibration pool and select τ on a separate part, using a bound on accepted-prediction error rather than accuracy alone.

We choose the threshold with the largest calibration coverage among those that satisfy the predefined reliability criterion. If none qualifies, the system delegates every shot. We keep the fitted gate fixed during testing. The term dynamic refers to the shot-dependent choice between paths, it does not mean that the threshold changes online. The evaluation chapter defines the error target and the adjustment for searching several thresholds.

An accepted shot uses the neural logical-flip decision. A rejected shot passes its original detector-event vector to the reference decoder. We do not remove local errors or construct a residual syndrome before fallback. This preserves the reference decoder's original input and lets us compare both paths on the same prediction task.

The benefit depends on coverage and execution cost together. A gate that accepts no shots saves no reference work, while a permissive gate may accept too many incorrect predictions. Even reliable coverage helps latency only when the avoided fallback cost exceeds the neural and gating overhead. We therefore evaluate delegation as part of the complete classical prediction pipeline, not only as a classification threshold.

5 Implementation

The main task is to connect the experimental records, their detector-error graphs, and the neural predecoder in one pipeline. We also implement model compression and confidence gating, so that we can evaluate their effects using the same data and measurement procedure.

5.1 Software architecture and execution environment

We implement the prototype in Python. Stim reads the circuit and detector error model descriptions [6], while NumPy handles the packed records, shot indices, and graph arrays. TensorFlow provides the tensor operations and automatic differentiation, and we use Keras to define the neural network, optimizer, and training callbacks. Separate evaluation tools compute prediction metrics, measure execution time, and export tables and figures. An optional PyMatching interface runs a reference decoder when a suitable detector error model is available.

We separate graph construction from shot processing. The circuit and DEM supply information that remains constant within one sample, whereas the detector-event vector changes from shot to shot. This allows us to prepare each graph once and reuse it throughout training and inference. The model shares its learned weights across samples, but each batch carries the graph belonging to its own records.

A single configuration file controls the complete experiment. The runner first trains the models, evaluates compression candidates on validation data, and fits their confidence gates. It then fixes the eligible models and gates before evaluating test predictions and measuring runtime. Each stage writes its results to the experiment directory. This separation lets us inspect a completed stage or repeat an interrupted one without rerunning all previous work.

5.2 Data-loading pipeline

The loader visits all 53 sample directories and selects the same surface-code configuration in each one. For every sample it requires four files:

```
circuit_ideal.stim
detection_events.b8
obs_flips_actual.b8
dems/correlations.dem
```

The circuit and DEM define the graph. The detector records supply the model inputs, and the observable-flip records supply the labels.

Stim's `b8` format packs detector or observable bits into bytes, with padding at the end of a shot when necessary [12]. The collection loader memory-maps the packed detector files and expands only the records needed for the current batch. This avoids keeping the much larger node-feature tensors for every shot in memory. Before training, the loader checks the record lengths against the circuit dimensions and verifies that detector records and labels contain the same number of shots.

We assign every shot a global index and retain its sample name and local row number. These identifiers keep predictions aligned with their labels and with any stored reference-decoder predictions. The partitioning step reserves separate training, validation, calibration, and test subsets within each sample. We save their identifiers once and reuse them for all experimental conditions. Only training records participate in the training shuffle.

Each batch contains shots from one sample. Samples can have different DEMs and therefore different graph sizes, even when they describe the same circuit configuration. Grouping records in this way lets us use the appropriate graph without padding unrelated graphs to a common size. After selecting a batch, the loader combines its detector events with the static node features. It passes the logical-flip labels separately to the training step, so they never enter the input features.

5.3 Graph creation

We use the circuit and DEM to construct the bipartite representation introduced in the design chapter. Stim expands repeated DEM blocks and resolves detector offsets before we process the error instructions. The graph builder checks that the circuit and DEM agree on their detector and logical-observable counts. It then creates one node for each detector and one node for each error instruction.

Two details require careful approach. First, detector annotations in this dataset can contain several spatial-temporal tuples. We interpret them according to the dataset convention and retain the representative detector position, its time slice, and the associated measurement metadata. Second, DEM targets follow modulo-two semantics. Repeating a target twice cancels its contribution. We therefore combine repeated detector and observable targets by parity.

For message passing, we connect each error node to the detectors in its support, export both directions of each connection, and add self-loops. We store the resulting connectivity as edge indices instead of a dense adjacency matrix. The preprocessing step also computes degree-based normalization coefficients. These coefficients control aggregation and remain separate from the mechanism probabilities supplied by the DEM.

The graph archive contains the connectivity and 13 node-feature channels. Static features describe the node type, coordinates, temporal position, mechanism probability, and boundary or logical support. One channel holds the detector-event value for the current shot. We leave that channel zero on error nodes because the recordings do not reveal which individual error mechanisms occurred. A collection of graph archives preserves the sample-specific structure and allows us to reconstruct the model inputs when loading saved weights.

5.4 Neural-network-based predecoder implementation

We implement the predecoder as a custom Keras model. Its forward pass consists of an input projection, residual graph-convolution blocks, a graph-level readout, and a fully connected prediction head. For a batch of B shots on a graph with N nodes, the feature tensor has shape $(B, N, 13)$. The model also receives the graph's edge indices and aggregation coefficients. This interface allows one set of learned weights to process graphs with different numbers of nodes and edges.

5.4.1 Graph convolutions

The graph-convolution layer first applies a learned linear transformation to the node features. It then gathers the transformed source-node values for each directed edge, multiplies them by the corresponding normalization coefficient, and sums incoming messages at their target nodes. TensorFlow's `unsorted_segment_sum` provides the aggregation operation.

The input projection establishes a common hidden width, which every convolution block preserves. Within a block, we add the convolution output to the previous node representation, then apply layer normalization and dropout. The two node types share the same transformations. Their feature channels and connectivity distinguish their roles. During validation and inference we disable dropout through the Keras training flag.

Although edge indices avoid a dense adjacency matrix, the layer still constructs an intermediate tensor with one message per edge and shot. Its size grows with the batch size, edge count, and hidden width. This is an important implementation cost when comparing the training batch sizes and the pruned models.

5.4.2 Graph-level prediction

The readout combines mean pooling, maximum pooling, and attention-weighted pooling over both node types. For the attention component, a dense layer assigns a score to each node and a softmax normalizes the scores across the graph. We concatenate the three pooled vectors before passing them to the prediction head. The attention weights affect this readout only, they do not replace the normalization coefficients inside the convolution blocks.

The final sigmoid layer returns one probability per shot, giving an output shape of $(B, 1)$. We keep this output in `float32`, including for the mixed-precision variant. It estimates whether the selected logical observable has flipped. The confidence gate uses this probability to decide whether to accept the neural prediction or invoke the fallback decoder.

5.4.3 Training and validation

We use custom Keras training and validation steps to control the loss and gradient computation. A training step runs the forward pass inside `tf.GradientTape`, computes binary cross-entropy, and applies gradients with Adam [11]. A validation step performs the same prediction and loss calculation with dropout disabled and without updating weights. We trace these steps with `tf.function`; their input signatures allow variable batch and graph sizes.

The model tracks loss, binary accuracy, and ROC AUC during training. Its loss tracker averages the scalar losses from successive batches, so a final short batch contributes one batch value. For the final predictive evaluation, we instead compute BCE over

individual saved predictions. This gives every test shot equal weight, regardless of its batch size.

5.5 Training and experiment management

The collection training loop supports both fixed-dataset and fresh-data schedules. In fixed mode, the runner selects a reproducible subset balanced across samples, and every epoch visits that subset in a new order. In fresh mode, successive epochs consume disjoint parts of a shuffled pool. The loop keeps the final partial portion and stops when it exhausts the available recordings. It never wraps around or substitutes simulated shots. Explicit epoch boundaries prevent an input prefetcher from consuming the next epoch's fresh records early.

We use Keras callbacks for validation-loss early stopping and checkpointing. Early stopping restores the weights selected by its minimum-improvement criterion. We also retain the checkpoint with the smallest observed validation loss, the two selections can differ because the checkpoint does not apply the same minimum-improvement threshold. Subsequent evaluation uses the restored model consistently. The loop saves loss histories and elapsed time after each completed epoch, together with the training-shot exposure.

The experiment runner reads `benchmark_config.py` and calls the training, compression, calibration, and evaluation functions directly. It records the configuration and stage status in a manifest. Separate worker processes release TensorFlow allocations between stages. A parent process enforces the invocation's time budget, while progress displays show the current stage and completed work. On resumption, the runner skips completed stages and restarts an unfinished stage from its original initialization or source model. Partial checkpoints remain available for inspection, but do not count as completed experimental conditions.

Before test evaluation, the runner fixes all validation-eligible model identities and their gate files. It verifies these identities again on resumption. Hashes of the input files, graph archives, weights, and partitions help detect changes that would make an earlier result inconsistent with the current run. Chapter 6 gives the exact configuration and the comparisons that this workflow supports.

5.6 Confidence-gated inference

We implement confidence gating as a separate stage after neural inference. The calibration routine splits the reserved calibration shots into two parts: one fits the temperature-scaling parameter, and the other selects the acceptance threshold. This keeps network training, probability calibration, and threshold selection separate. The gate file stores its parameters, calibration-shot identifiers, and model identity.

During inference, the gate calibrates the predicted probability and compares its confidence with the saved threshold. An accepted prediction supplies the logical-flip decision directly. For a rejected shot, the live fallback receives the original full detector-event vector. We can also combine neural and stored reference predictions for a quality comparison without executing a decoder. If calibration supports no threshold at the requested error target, the gate rejects every shot. We implement no density-dependent threshold in this version.

The prototype predicts one logical-observable flip. Connecting this output to a quantum controller would require a mapping to the relevant Pauli-frame component and an interface for applying it. The present implementation stops at the classical prediction and delegation stage.

5.7 Model compression

We implement structured pruning by constructing a smaller network and copying selected weights into it. The pruning routine ranks hidden channels by the summed absolute magnitudes of their associated parameters. It retains the same channel indices throughout the residual stack and adjusts the projection, convolutions, normalization, attention input, and readout inputs together. The prediction head has its own unit selection. These coordinated changes preserve compatible tensor dimensions while reducing the intermediate message width. The graph and its input features remain unchanged.

We fine-tune the smaller model on a fixed training subset with the configured learning rate and validation-loss early stopping. This fine-tuning uses repeated data passes for both initial training schedules. We then compare its validation BCE with that of its direct parent to decide whether it enters final evaluation. The mixed-precision candidate starts from the eligible pruned model and requires no additional training.

For mixed precision, the projection, feature transformations, and hidden dense layers compute in `float16`. Explicit casts keep edge coefficients, message accumulation,

residual representations, normalization, pooling, and output probabilities in `float32`. We export transformation parameters at reduced precision and reload the exported representation before assessing it. This checks the effect of stored rounding as well as the computation policy. Keras can retain variables in `float32` under a mixed-precision policy [13], so the evaluation distinguishes exported weight size from resident parameter storage.

5.8 Evaluation and benchmarking

The evaluation tools reload a saved model and its graph collection, predict the requested held-out shots, and compare the decisions with their labels. For reference comparisons, they check the shot identifiers and label alignment before combining predictions. They also include a constant predictor whose class comes from the training labels. The exporter collects prediction, confidence, resource, and runtime results in CSV files and creates the LaTeX tables and diagnostic figures used in the next chapter.

The follow-up reference comparison reads the saved probability arrays, fitted temperatures, partitions, and classical-decoder predictions. It checks their identifiers and hashes before evaluating a configured list of confidence thresholds. For each threshold, it combines accepted neural decisions with the reference decisions on the remaining shots, and counts the reference errors corrected and new errors introduced. It exports pooled and per-sample results, conservative paired uncertainty bounds, and static accuracy–coverage plots. This workflow reuses the completed experiments without retraining the models or repeating inference timing.

For runtime measurements, we load the model, select a sample's graph, and prepare the input before timing. We trace separate functions for the model alone, detector events through prediction, and detector events through the gate. Warm-up calls precede each latency measurement. Every timed neural call includes materializing its output on the host, so the timer cannot stop while the GPU is still computing the required result. GPU predictions must also agree with a CPU reconstruction within the configured numerical tolerance, and gate decisions must agree exactly.

We measure throughput and memory separately from single-shot latency. Progress messages appear around complete timing stages rather than inside the measured calls. The saved report records timing samples, device placement, software versions, and hardware details.

6 Evaluation

We evaluate prediction quality, reliable delegation, and the cost of neural inference. The completed experiments cover five, nine, and 25 rounds of error correction. Each uses the same training settings and compares twelve model variants. The five-round models improve on the constant predictor, while the nine-round models return the majority-class decision on every test shot. At 25 rounds, accuracy ranges from 49.93% to 50.08%, with AUCs close to 0.5. All fitted gates delegate every shot at the chosen reliability target. A follow-up comparison at five and nine rounds with stored correlated-matching predictions examines how other confidence thresholds change accuracy and reference-decoder work. We examine these outcomes through the three research questions below. Section 6.5 brings together the findings across round counts.

6.1 Methodology

6.1.1 Data and partitions

We use all 53 samples, from `sample_00` to `sample_52`, and select the distance-five, X-basis configuration at `d5_at_q6_5`. The completed studies use the `r05`, `r09`, and `r25` directories. Each study supplies test evaluations and timing measurements for all twelve eligible models. Table 6.1 records the available comparisons.

Table 6.1: Completed round-count comparisons. Each variant has test and timing results.

Rounds	Status	Variants	Test shots per variant
5	Complete	12	397,500
9	Complete	12	397,500
25	Complete	12	397,500

Each completed round-count study contains 75 000 shots per sample, giving 3 975 000 shots. Within each sample, we reserve 80% for training, 5% for validation, 5% for calibration, and 10% for testing. The pooled partitions contain 3 180 000, 198 750,

198 750, and 397 500 shots, respectively. Every sample contributes 7,500 test shots. We report prediction errors per complete experimental shot.

Split seed 73 determines the partitions, with the numeric sample suffix added for each local split. Within a round-count study, every variant uses the same test identifiers and labels. Across round counts, the records belong to different experimental configurations. We compare their aggregate performance and retain the constant predictor for each configuration as a reference for class imbalance.

Each sample supplies its own circuit, detector records, logical-flip labels, and correlation-derived DEM. We build its graph once and share the learned network parameters across samples. Table 6.2 shows the graph dimensions. Nine-round graphs contain about 67% more edges than five-round graphs, so each message-passing step handles a larger input. At 25 rounds, each graph contains 648 detector nodes, 10,655 error nodes, and 77,287 directed edges including self-loops. Its 11,303 nodes are more than four times the five-round count.

Table 6.2: Graph dimensions in the completed studies. Directed edges include self-loops.

Rounds	Detectors	Error nodes	All nodes	Edges
5	168	2495	2663	17767
9	264	4127	4391	29671
25	648	10655	11303	77287

The split evaluates unseen shots from samples represented in training. Transfer to new samples would require holding out complete samples. The supplied DEMs remain fixed throughout each comparison. Their original data selection is unknown, so the results are conditional on these priors.

6.1.2 Training and model selection

We compare fixed-data and fresh-data training with batch sizes 64 and 128. Each combination produces a baseline, a pruned candidate, and a mixed-precision version of the pruned model. All twelve candidates in each completed study pass the validation eligibility rule. Table 6.3 summarizes the shared configuration.

Fixed-data training revisits the same 120 000 shots in a new order each epoch. Fresh-data training draws disjoint portions from the reserved pool. This permits 26 full epochs and a final epoch of 60 000 shots, for a maximum of 27 epochs. Pruning uses a fixed

Table 6.3: Shared experimental settings. Epoch limits apply before early stopping.

Setting	Value
Training schedules and batch sizes	Fixed and fresh; 64 and 128
Initialization and split seeds	73
Baseline network	13 input features, 96 hidden channels, 3 GCN blocks
Prediction head and dropout	128/64 units; dropout 0.1
Baseline optimization	Adam, learning rate 10^{-4}, at most 100 epochs
Fixed training subset	120 000 shots across the 53 samples
Fresh-data exposure	Up to 120 000 unused shots per epoch
Pruned network	64 hidden channels; head 64/32
Pruning fine-tuning	Fixed 120 000-shot subset; 10^{-4}; at most 20 epochs
Early stopping	Validation loss; patience 15; minimum improvement 10^{-5}
Compression eligibility	Validation BCE increase at most 0.01 relative to the direct parent
Gate selection	Error target 0.01; $\alpha = 0.05$; 101 thresholds; at least 20 accepted shots
Latency measurement	Batch one, 50 warm-up calls and 500 timed calls per sample and scope
Throughput measurement	Batch eight; five passes over the timing inputs
Paired bootstrap	2 000 replicates

subset for fine-tuning in both groups. A pruned model therefore inherits its parent's exposure and adds repeated training on that subset. Mixed-precision conversion adds no training epochs.

Early stopping restores the weights selected by its minimum-improvement rule. The evaluation uses these restored weights. The separately retained minimum-loss checkpoint can correspond to another epoch because it records every decrease in validation loss. We record completed epochs and actual shot exposure, including training after the selected epoch.

We use one initialization seed. The comparisons describe the tested training conditions, and repeated initializations would be needed to estimate training variability. Before test evaluation, the runner fixes the eligible model identities and fitted gates. All three completed studies use an unlimited invocation duration. The manifests record stage times and interrupted or failed attempts.

6.1.3 Metrics

For test shot i, the network returns logical-flip probability $\widehat{p}_i$. We classify it using $\widehat{\ell}_i = \mathbf{1}[\widehat{p}_i > 0.5]$ and compare it with the recorded label ℓ_i. For N test shots,

$$\widehat{P}_{\text{fail}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\widehat{\ell}_i \neq \ell_i].$$

We report accuracy, incorrect-prediction counts, and ROC AUC. Binary cross-entropy measures probability quality:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \left[\ell_i \log \widetilde{p}_i + (1 - \ell_i) \log(1 - \widetilde{p}_i) \right],$$

where $\widetilde{p}_i$ clips $\widehat{p}_i$ to $[10^{-7}, 1 - 10^{-7}]$. We compute the test metrics from individual saved predictions, giving each shot equal weight.

The constant predictor chooses the majority class among the training shots consumed by the run. It predicts no logical flip at five and nine rounds, and a logical flip at 25 rounds. A paired bootstrap resamples neural and constant-predictor errors on the same shots. The reported intervals assume independent shots, correlations within a sample could affect their coverage.

The confidence-gate comparison uses coverage and error among accepted shots. For execution cost, we measure median, P95, and P99 latency, throughput, and memory. Parameter count, exported-array bytes, resident-variable bytes, process RSS, and TensorFlow allocator peaks describe different parts of the storage and memory cost.

6.1.4 Classical reference and threshold comparison

For the five- and nine-round studies, we use the dataset's stored correlated-matching predictions obtained with the DEM inferred from correlations. Within each sample, these predictions come from `obs_flips_predicted/dem_correlations/correlated_matching.b8`. We align them with the saved neural predictions using the sample name and local shot index, and check the labels and artifact hashes. This gives a paired comparison on the same records for each model and reference decoder.

After the original benchmark, we compare twelve fixed confidence thresholds: 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.97, 0.98, 0.985, 0.99, 0.995, and 1.0. We retain the fitted temperature for each model and compute confidence from the saved probabilities. The comparison reuses the completed models and contains no additional training or latency

measurements. It reports the calibration-selection and test subsets separately, keeping the temperature-fitting subset outside the comparison.

This sweep is an exploratory follow-up to the original frozen-gate evaluation. We report all configured cutoffs and preserve the original gate as a separate policy. Any operating threshold chosen from these results would need confirmation on new held-out data. The exported calibration probabilities and the original gate-fitting inference pass give slightly different counts near some cutoffs: the largest differences are three accepted shots and one error. We retain the original gate selection outcome and use the exported predictions consistently for the follow-up comparison.

6.1.5 Hardware and timing procedure

We run the benchmark on an NVIDIA H200 NVL inside a Linux Docker container. All three studies record Python 3.11.13, TensorFlow 2.20.0, NumPy 1.26.4, a TensorFlow CUDA build version of 12.5.1, and cuDNN major version 9. The reports leave the exact driver, compiler executable, CPU model, TF32 setting, and source revision unspecified. Those details remain part of the execution record needed for reproducing the environment.

We time three scopes: model-only inference, detector events through prediction, and detector events through probability calibration and the gate. Inputs start in host memory, and every timed call waits for the result to materialize on the host. We load the model, select the sample graph, and trace the function before timing. The measured path covers classical processing of a complete detector history.

For each sample and scope, we cycle through 32 sampled test shots during 500 timed calls after 50 warm-up calls. Pooling the samples gives 26,500 timing observations from 1,696 distinct records. Throughput uses separate batches of eight shots and five passes over the inputs. We calculate it as completed shots divided by elapsed time.

The benchmark checks sampled GPU predictions against a CPU reconstruction. The saved relative and absolute tolerances are 10^{-5} and 10^{-6} for `float32`, and 10^{-3} and 10^{-4} for mixed precision. All completed timing stages pass these checks. Separate memory runs record TensorFlow allocator peaks and process RSS sampled every 5 ms. This sampling interval can miss brief process-memory peaks.

6.2 Prediction quality and training conditions

The predecoder needs to learn information that helps predict logical flips. We compare its predictions with the training-majority baseline and correlated matching, then examine the effects of the training schedule and batch size.

Research question 1: How do the training schedule and batch size affect logical-flip prediction quality and training cost?

6.2.1 Prediction results at five, nine, and 25 rounds

Tables 6.4, 6.5, and 6.6 report all eligible variants. At five rounds, the constant predictor reaches 55.27% accuracy. Every neural baseline improves on it. The fixed-data, batch-64 baseline reaches 62.31% accuracy, BCE 0.6187, and AUC 0.6908. Its error rate is 7.03 percentage points lower than the constant predictor's, with a paired-bootstrap 95% interval of $[-0.0721, -0.0684]$ for the error-rate difference. Its per-sample accuracy ranges from 60.33% to 64.75%, and it improves on the constant predictor in all 53 samples.

Table 6.4: Prediction quality at 5 rounds on 397,500 held-out shots. The constant predictor reaches 55.27% accuracy. Batch denotes training batch size.

Schedule	Batch	Variant	Accuracy (%)	BCE	AUC
Fresh	64	Baseline	58.07	0.6501	0.6360
Fresh	64	Pruned	59.33	0.6389	0.6501
Fresh	64	Mixed	59.33	0.6389	0.6502
Fresh	128	Baseline	58.58	0.6459	0.6373
Fresh	128	Pruned	59.17	0.6404	0.6470
Fresh	128	Mixed	59.17	0.6404	0.6470
Fixed	64	Baseline	62.31	0.6187	0.6908
Fixed	64	Pruned	62.32	0.6184	0.6904
Fixed	64	Mixed	62.31	0.6184	0.6904
Fixed	128	Baseline	60.04	0.6361	0.6614
Fixed	128	Pruned	60.08	0.6368	0.6617
Fixed	128	Mixed	60.08	0.6369	0.6617

The five-round fresh-data baselines reach 58.07% accuracy at batch size 64 and 58.58% at batch size 128. The fixed-data, batch-128 baseline reaches 60.04%. The fixed-data, batch-64 condition therefore gives the strongest baseline result in this experiment. The exposure comparison below helps interpret this ordering.

Table 6.5: Prediction quality at 9 rounds on 397,500 held-out shots. The constant predictor reaches 51.45% accuracy. Batch denotes training batch size.

Schedule	Batch	Variant	Accuracy (%)	BCE	AUC
Fresh	64	Baseline	51.45	0.6927	0.5096
Fresh	64	Pruned	51.45	0.6927	0.4916
Fresh	64	Mixed	51.45	0.6927	0.4916
Fresh	128	Baseline	51.45	0.6927	0.4991
Fresh	128	Pruned	51.45	0.6927	0.5070
Fresh	128	Mixed	51.45	0.6927	0.5071
Fixed	64	Baseline	51.45	0.6927	0.4930
Fixed	64	Pruned	51.45	0.6927	0.4971
Fixed	64	Mixed	51.45	0.6927	0.4972
Fixed	128	Baseline	51.45	0.6928	0.5030
Fixed	128	Pruned	51.45	0.6927	0.4961
Fixed	128	Mixed	51.45	0.6927	0.4961

Table 6.6: Prediction quality at 25 rounds on 397,500 held-out shots. The constant predictor reaches 49.93% accuracy. Batch denotes training batch size.

Schedule	Batch	Variant	Accuracy (%)	BCE	AUC
Fresh	64	Baseline	49.93	0.6932	0.4997
Fresh	64	Pruned	49.93	0.6932	0.5002
Fresh	64	Mixed	49.93	0.6932	0.5002
Fresh	128	Baseline	49.93	0.6932	0.5016
Fresh	128	Pruned	49.93	0.6932	0.4995
Fresh	128	Mixed	49.93	0.6932	0.4994
Fixed	64	Baseline	50.08	0.6931	0.5021
Fixed	64	Pruned	49.93	0.6932	0.5016
Fixed	64	Mixed	49.93	0.6932	0.5016
Fixed	128	Baseline	49.95	0.6932	0.5017
Fixed	128	Pruned	49.93	0.6932	0.5008
Fixed	128	Mixed	49.93	0.6932	0.5008

At nine rounds, all twelve variants reach 51.45% accuracy and make 192,987 errors on 397,500 test shots. Every variant predicts no logical flip on every test shot, matching the constant predictor's decisions. BCE remains close to 0.6927, and AUC ranges from approximately 0.492 to 0.510. The probabilities have small variations, which account for the AUC differences despite identical binary decisions.

At 25 rounds, accuracy ranges from 49.93% to 50.08%, BCE remains close to 0.6932,

and AUC ranges from 0.4994 to 0.5021. Ten variants predict a logical flip on every test shot, matching the training-selected constant predictor and making 199,016 errors. The two fixed-data baselines produce both decisions. Batch size 64 reaches 50.08% accuracy and AUC 0.5021, while batch size 128 reaches 49.95% accuracy. The batch-64 baseline removes a net 584 errors relative to the constant predictor, an accuracy gain of 0.147 percentage points. Its paired-bootstrap 95% interval for the error-rate difference is $[-0.00250, -0.00048]$. This small gain leaves accuracy close to chance on the nearly balanced test set.

Table 6.7 places these results alongside the classical decoder. Correlated matching reaches 82.11% accuracy at five rounds and 73.19% at nine rounds. It makes 71,111 and 106,558 errors, respectively, on the corresponding 397,500 test shots. It outperforms every neural variant in both studies. The five-round network therefore learns information beyond the class imbalance, but its overall accuracy remains substantially below that of the reference decoder. Its potential use as a predecoder depends on the quality of the subset accepted by the gate, which we examine in Section 6.3.

Table 6.7: Correlated matching and the highest neural test accuracy for each round count. Both use the same shots within each study.

Rounds	Test shots	Classical errors	Classical (%)	Neural (%)
5	397,500	71,111	82.11	62.32
9	397,500	106,558	73.19	51.45

The common settings produce substantially different outcomes at five rounds and the two longer histories. Longer histories enlarge the graph and change the prediction task. The nine- and 25-round results expose a learning limitation of this architecture and training procedure in the tested setting. Determining its cause would require additional training experiments. In particular, the present single-seed comparison leaves the roles of optimization, network capacity, and graph representation unresolved.

6.2.2 Training exposure and duration

Table 6.8 records unique shots, total presentations, completed epochs, and training-stage time. These quantities help distinguish data reuse from the amount of optimization.

Both five-round fresh-data baselines exhaust the pool after 27 epochs. The fixed-data, batch-64 baseline completes 100 epochs, revisiting its 120 000 shots for 12 000 000 presentations. The fixed-data, batch-128 baseline completes 84 epochs, giving 10 080 000 presentations. Thus, the stronger fixed-data results also use more training presentations.

Table 6.8: Baseline training exposure and stage duration, including loading and validation. Presentations count repeated uses of a shot and are given in millions.

Rounds	Schedule	Batch	Unique shots	Epochs	Presentations	Minutes
5	Fresh	64	3,180,000	27	3.18	78.3
5	Fresh	128	3,180,000	27	3.18	84.9
5	Fixed	64	120,000	100	12.00	283.2
5	Fixed	128	120,000	84	10.08	167.2
9	Fresh	64	3,180,000	27	3.18	91.8
9	Fresh	128	2,640,000	22	2.64	58.5
9	Fixed	64	120,000	23	2.76	87.3
9	Fixed	128	120,000	19	2.28	51.2
25	Fresh	64	2,760,000	23	2.76	153.3
25	Fresh	128	3,180,000	27	3.18	158.7
25	Fixed	64	120,000	20	2.40	134.2
25	Fixed	128	120,000	20	2.40	120.3

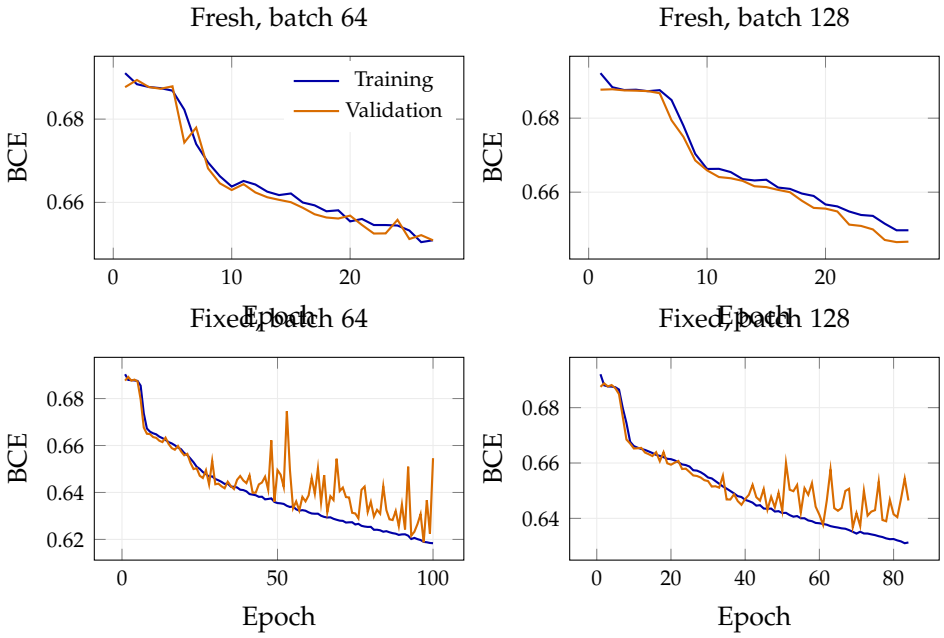


Figure 6.1: Training and validation BCE for the four 5-round baseline conditions. Each panel uses its own axis ranges.

A comparison with matched update counts or elapsed time would help separate these effects.

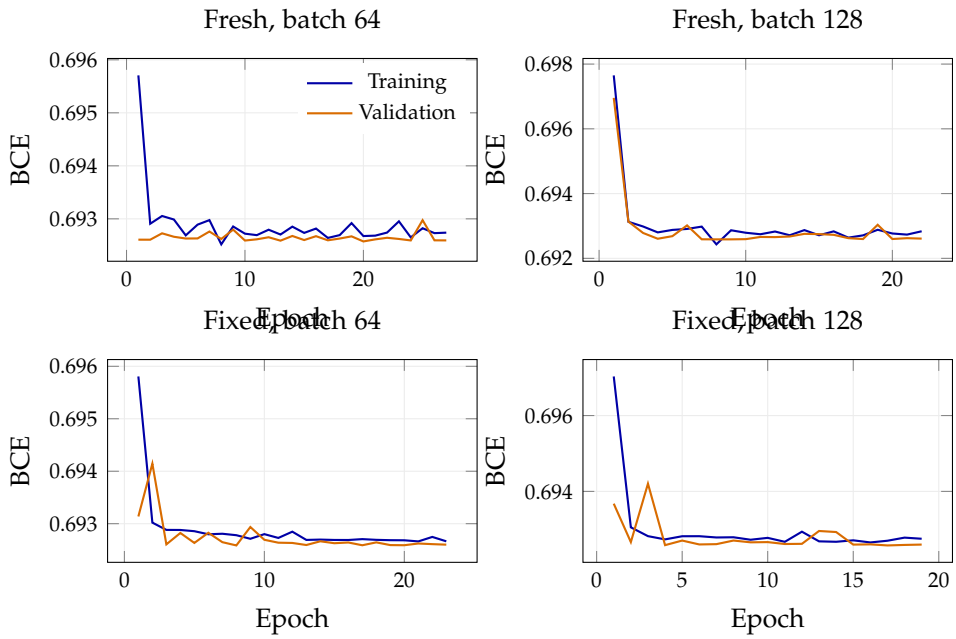


Figure 6.2: Training and validation BCE for the four 9-round baseline conditions. Each panel uses its own axis ranges.

At nine rounds, the fresh-data, batch-64 baseline completes 27 epochs and consumes all 3 180 000 reserved training shots. Batch size 128 stops after 22 epochs and 2 640 000 unique shots. The fixed-data baselines stop after 23 and 19 epochs. Their shorter runs reflect early stopping as validation loss levels off. Figures 6.1 and 6.2 show the loss histories: the five-round curves continue to improve, while the nine-round curves settle close to the loss of a nearly constant prediction.

At 25 rounds, the fresh-data, batch-64 baseline completes 23 epochs and uses 2 760 000 unique shots. The fresh-data, batch-128 baseline exhausts the pool in 27 epochs. Both fixed-data baselines stop after 20 epochs, giving 2 400 000 presentations of their 120 000-shot subsets. Figure 6.3 shows validation loss close to 0.6932 throughout these runs. Additional exposure in the fresh-data conditions does not produce higher test accuracy in this study.

The five-round baseline stages take 78.3-283.2 minutes, the nine-round stages take 51.2-91.8 minutes. Stage duration includes loading and validation. The recorded study invocations total 16.37 hours for five rounds, including failed attempts and resumption, and 11.33 hours for nine rounds. Both exceed the original ten-hour budget. The shorter nine-round study largely reflects fewer completed training epochs. The

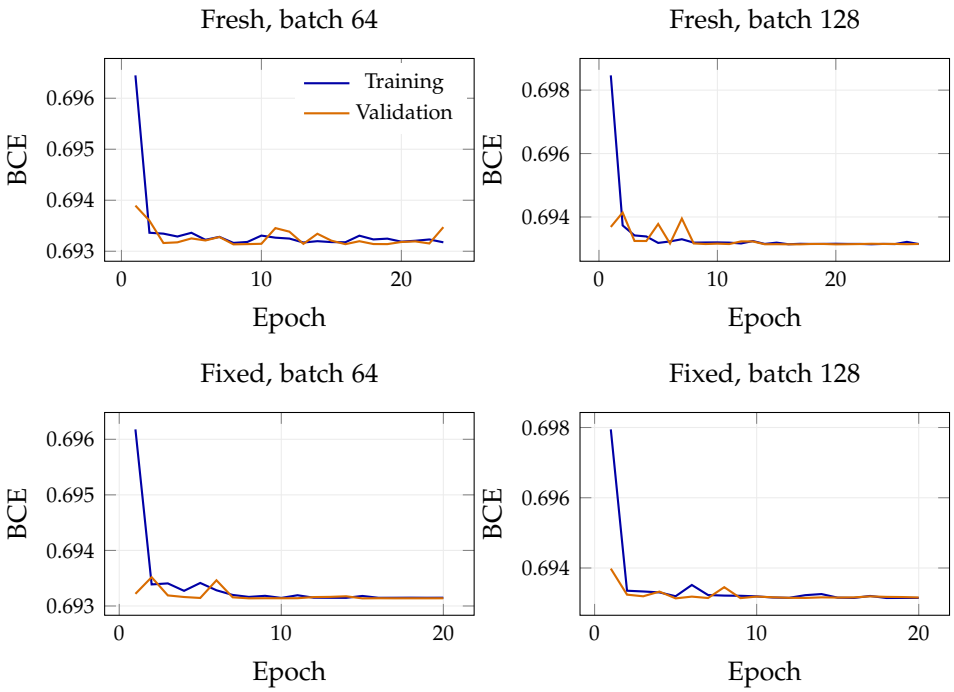


Figure 6.3: Training and validation BCE for the four 25-round baseline conditions. Each panel uses its own axis ranges.

25-round baseline stages take 120.3–158.7 minutes. One invocation in its manifest lacks an elapsed duration, so we cannot reconstruct the complete study runtime from the recorded invocation times.

6.3 Confidence gating and delegation

A useful gate must identify enough reliable predictions to reduce the reference decoder's workload. We first assess the gate fixed on calibration data, then compare alternative thresholds with correlated matching on the same accepted shots.

Research question 2: Can the confidence gate reduce the fraction of shots sent to a reference decoder while retaining reliable logical-flip predictions?

6.3.1 Calibration and selection

For each model, we fit temperature scaling on 99,375 calibration shots and select the threshold on the remaining 99,375. Temperature scaling uses

$$q_i = \sigma\left(\frac{\operatorname{logit}(\widetilde{p}_i)}{T}\right), \qquad 0.05 \le T \le 20.$$

The gate accepts a shot when $a_i = \mathbf{1}[\max(q_i, 1-q_i) \ge \tau]$. We search 101 equally spaced thresholds in $[0.5, 1]$ and select the candidate with the largest coverage whose one-sided binomial error upper bound is at most 0.01. A qualifying candidate must accept at least 20 selection shots. The Bonferroni adjustment divides $\alpha = 0.05$ by the number of candidates [14]. The adjustment covers threshold selection within each fitted model.

On the test set, coverage is $C = N_{\text{accepted}}/N$, and accepted-shot error is

$$\widehat{P}_{\text{fail|accept}} = \frac{\sum_i a_i \mathbf{1}[\mathbf{1}[q_i > 0.5] \ne \ell_i]}{\sum_i a_i}.$$

We leave this quantity undefined when the accepted set is empty. The binomial bounds assume independent shots and describe the pooled set of samples. With zero observed errors among n accepted shots, the unadjusted one-sided 95% upper bound is

$$p_{\text{upper}} = 1 - 0.05^{1/n}. \tag{6.1}$$

6.3.2 Gate outcomes

Every fitted gate in the five-, nine-, and 25-round studies is disabled before test evaluation because no candidate threshold meets the selection criterion. Coverage is zero for all 36 variants (Table 6.9). Each model delegates all 397,500 test shots to the reference path.

Table 6.9: Frozen confidence gates at the 1% accepted-error target. The coverage column gives the range across models within each round count. Accepted-error ranges use nonempty accepted sets.

Rounds	Models	Enabled gates	Coverage (%)	Accepted error (%)
5	12	0	0.00	Undefined
9	12	0	0.00	Undefined
25	12	0	0.00	Undefined

For the five-round, fixed-data, batch-64 baseline, the smallest calibration upper bound among candidates accepting at least 20 shots is 3.19%. At $\tau \approx 0.985$, that candidate

accepts 1,268 selection shots and makes 21 errors. Its observed error rate of 1.66% already exceeds the 1% target. The selection rule therefore rejects it.

For the corresponding nine-round baseline, the smallest upper bound among candidates accepting at least 20 shots is about 49.10%. At threshold 0.5, it accepts all 99,375 selection shots and makes 48,272 errors. Its nearly constant output gives the threshold little ability to separate reliable and unreliable records.

At 25 rounds, the smallest error upper bound among candidates accepting at least 20 shots is 50.41% for the fixed-data, batch-64 baseline. This candidate accepts all 99,375 selection shots and makes 49,579 errors at threshold 0.5. It also falls well short of the 1% target. All twelve variants accept zero test shots, so accepted-shot error remains undefined for the entire 25-round group.

At the original 1% accepted-error target, the fitted gates save zero reference-decoder calls. A serial implementation would process every shot through the neural model and gate before invoking the reference decoder. Its expected time at zero coverage would be

$$\mathbb{E}[T_{\text{hybrid}}] = \mathbb{E}[T_{\text{neural}} + T_{\text{gate}}] + \mathbb{E}[T_{\text{reference}}].$$

With these disabled gates, the hybrid returns the reference decoder's predictions and therefore has the same accuracy. The stored reference outputs also let us examine hybrid quality at other thresholds. Measuring relative execution speed still requires live reference and hybrid timings.

6.3.3 Accuracy and reference-decoder workload

For a fixed threshold, we use the neural decision on accepted shots and the correlated-matching decision on all remaining shots. Let H count accepted shots for which the neural prediction is wrong and the reference prediction is correct. Let B count accepted shots for which the neural prediction is correct and the reference prediction is wrong. Then

$$E_{\text{hybrid}} = E_{\text{reference}} + H - B, \qquad \Delta\text{accuracy} = 100\frac{B-H}{N} \quad \text{percentage points.}$$

This paired comparison accounts for the errors that both decoders can make. The hybrid preserves or improves the observed accuracy when $B \geq H$. Coverage gives the fraction of reference-decoder calls avoided. The time saved also depends on the reference cost of those accepted shots and the neural processing required for every shot.

We quantify uncertainty by bounding the two discordance probabilities p_H and p_B with Clopper–Pearson binomial intervals [15]. From bounds $[L_H, U_H]$ and $[L_B, U_B]$, the error-rate difference has the conservative interval $[L_H - U_B, U_H - L_B]$. The report divides $\alpha = 0.05$ across four tails and 5,520 comparison opportunities, covering both subsets, all models, the requested cutoffs, the two control policies, and the original gate candidates. Each tail uses probability $0.05/(4 \cdot 5520)$. These simultaneous bounds assume independent shots, they leave within-sample dependence and distribution shift unresolved. An interval containing zero permits both an accuracy gain and a loss. Evidence for preserved accuracy would require its error-difference upper bound to be at most zero, or a justified degradation margin fixed in advance.

Table 6.10 and Figure 6.4 show the five-round, fixed-data, batch-64 baseline. Raising the threshold reduces coverage and increases accuracy among accepted neural predictions. Correlated matching also becomes more accurate on this subset. For example, at $\tau = 0.9$, the neural model reaches 95.14% accepted-shot accuracy, while correlated matching reaches 96.57% on those same records. Accepting 5.867% of shots introduces 550 errors and corrects 216, giving 334 extra errors overall. Hybrid accuracy falls from 82.11044% to 82.02642%.

Table 6.10: Fixed-threshold test comparison at 5 rounds for the fixed-data, batch-64 baseline. Coverage equals the fraction of reference calls avoided.

τ	Coverage (%)	NN acc. (%)	Ref. acc. (%)	Hybrid acc. (%)	Δ acc. (pp)
0.5	100.000	62.31	82.11	62.30591	-19.80453
0.6	41.571	75.86	88.04	77.04855	-5.06189
0.7	24.758	83.88	91.17	80.30415	-1.80629
0.8	14.122	90.07	94.01	81.55447	-0.55597
0.9	5.867	95.14	96.57	82.02642	-0.08403
0.95	3.439	96.58	97.52	82.07799	-0.03245
0.97	2.446	97.16	97.89	82.09258	-0.01786
0.98	1.749	97.91	98.46	82.10088	-0.00956
0.985	1.284	98.37	98.73	82.10591	-0.00453
0.99	0.678	98.66	98.89	82.10893	-0.00151
0.995	0.108	99.53	99.77	82.11019	-0.00025
1	0.000	—	—	82.11044	+0.00000

NN and reference accuracies refer to the same accepted subset. Hybrid accuracy uses all test shots; its change is relative to correlated matching alone. A dash denotes an empty accepted subset.

The threshold $\tau = 0.985$ connects the follow-up comparison with the calibration example above. On the 1,268 accepted calibration-selection shots, the neural model makes 21 errors and correlated matching makes 15. The hybrid corrects four reference

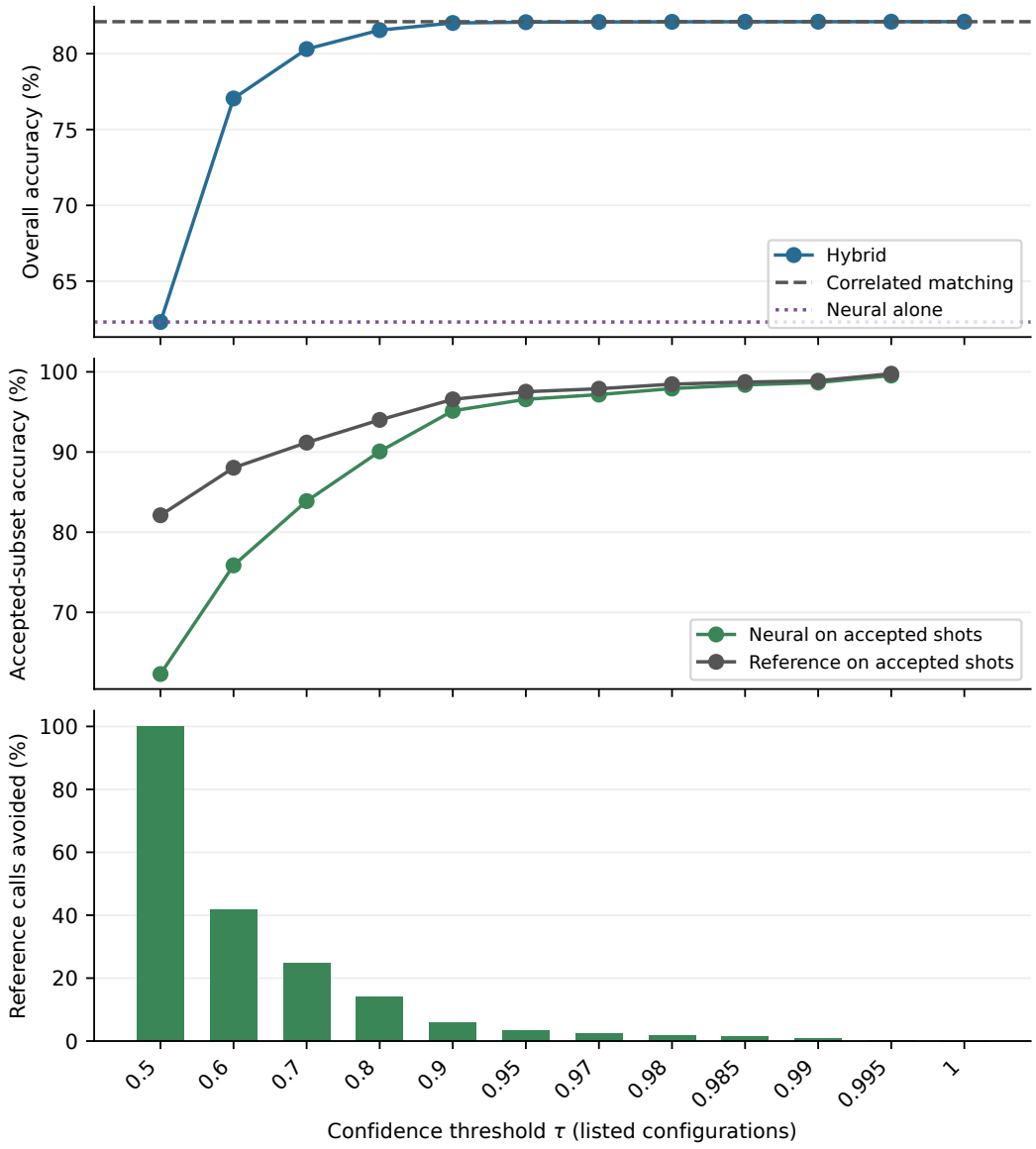


Figure 6.4: Confidence-threshold comparison for the five-round, fixed-data, batch-64 baseline. The panels show overall accuracy, the accuracy of both decoders on the accepted subset, and the fraction of reference calls avoided. Thresholds occupy equally spaced positions for readability. Accepted-subset accuracy is undefined at threshold 1 because no test shots qualify. Table 6.10 resolves the small accuracy differences at high thresholds.

errors and introduces ten, giving six additional errors. On the test set, this threshold accepts 5,103 shots, or 1.284%. The neural model makes 83 errors on that subset and the reference makes 65. Of their disagreements, 15 favour the neural model and 33 favour the reference. The resulting 18 additional errors reduce overall accuracy by 0.00453 percentage points, to 82.10591%. The simultaneous interval for the accuracy change is approximately $[-0.01619, +0.00715]$ percentage points. This small observed loss and its interval do not establish that the hybrid preserves accuracy.

At $\tau = 0.995$, coverage falls to 0.108%, and the hybrid introduces one extra test error. Across all twelve five-round models and twelve cutoffs, 120 configurations accept at least one shot. Of these, 111 reduce the observed accuracy and nine leave it unchanged. The largest accepted set among those nine contains 34 shots, or 0.00855% of the test set. The neural and reference decisions agree on all 34. No positive-coverage configuration has an error-difference upper bound at or below zero.

At nine rounds, threshold 0.5 accepts every shot. All twelve models then give the neural-only accuracy of 51.45%, compared with 73.19% for correlated matching. Every configured threshold of 0.6 or higher accepts zero shots. The grid therefore finds no intermediate workload reduction at nine rounds. Additional cutoffs between 0.5 and 0.6 would resolve the transition more closely.

The follow-up answers the second research question in terms of relative accuracy as well as the original absolute error target. The five-round model identifies increasingly reliable subsets, but correlated matching also handles those subsets well. The tested policies provide no evidence of a useful reduction in reference calls with preserved overall accuracy. Their coverage quantifies avoided calls, a claim about execution-time savings requires the corresponding runtime measurements.

6.4 Compression, latency, and memory

Compression changes both the number of parameters and the operations performed during inference. We compare the baseline, pruned, and mixed-precision models within each training condition.

Research question 3: How do structured pruning and mixed precision affect prediction quality, gate coverage, execution latency, and memory use?

6.4.1 Prediction quality and parameter storage

Pruning reduces hidden width from 96 to 64 and the prediction head from 128/64 to 64/32 units. The three graph-convolution blocks and input graph remain fixed. We admit each compressed model when its validation BCE exceeds its direct parent's by at most 0.01.

At five rounds, pruning and fine-tuning improve fresh-data accuracy from 58.07% to 59.33% for batch size 64 and from 58.58% to 59.17% for batch size 128. These changes reflect the combined pruning and fine-tuning procedure. An additional baseline with the same fine-tuning budget would help isolate the contribution of channel removal.

The five-round fixed-data models change little. At batch size 64, pruning corrects a net 42 additional test shots, taking accuracy from 62.3059% to 62.3165%. The largest absolute accuracy difference between a mixed-precision model and its pruned parent is about 0.0033 percentage points. At nine rounds, every compressed variant retains the constant binary decision. At 25 rounds, pruning reduces fixed-data, batch-64 accuracy from 50.08% to 49.93%, removing its small gain over the constant predictor. All eight compressed 25-round variants predict a logical flip on every shot. The original fitted gates retain zero coverage in all three studies.

Table 6.11: Parameter count and storage, common to the completed round counts. Payload counts exported arrays; resident storage counts loaded variables.

Variant	Parameters	Payload (KiB)	Resident (KiB)
Baseline	75,266	294.01	294.01
Pruned	28,290	110.51	110.51
Mixed	28,290	56.57	110.51

Pruning removes 62.4% of the parameters, reducing the count from 75,266 to 28,290. The mixed export stores 57,928 parameter bytes, an 80.8% reduction from the baseline's 301,064 bytes. After loading, it holds 113,160 bytes of resident variables, matching the pruned `float32` model. The loader reconstructs these variables in single precision. The additional parameter-storage saving from mixed precision therefore applies to the exported arrays.

6.4.2 Latency and throughput

Tables 6.12, 6.13, and 6.14 report the detector-to-prediction path. Median single-shot latency lies between 4.39 and 4.83 ms across all three round counts. The 25-round

medians range from 4.50 to 4.83 ms.

Table 6.12: Detector-to-prediction execution at 5 rounds on the H200 NVL with host inputs. Latency uses batch one; throughput uses batch eight. Batch denotes training batch size.

Schedule	Batch	Variant	Median (ms)	P95 (ms)	P99 (ms)	Shots/s
Fresh	64	Baseline	4.435	4.793	5.071	1913
Fresh	64	Pruned	4.391	4.854	5.117	1963
Fresh	64	Mixed	4.830	5.424	5.601	1725
Fresh	128	Baseline	4.439	4.809	5.113	1930
Fresh	128	Pruned	4.439	4.827	5.069	1950
Fresh	128	Mixed	4.811	5.449	5.655	1738
Fixed	64	Baseline	4.493	4.790	5.053	1902
Fixed	64	Pruned	4.443	4.824	5.054	1942
Fixed	64	Mixed	4.704	5.318	5.449	1798
Fixed	128	Baseline	4.495	4.855	5.116	1874
Fixed	128	Pruned	4.411	4.841	5.116	1962
Fixed	128	Mixed	4.694	5.339	5.503	1830

Table 6.13: Detector-to-prediction execution at 9 rounds on the H200 NVL with host inputs. Latency uses batch one; throughput uses batch eight. Batch denotes training batch size.

Schedule	Batch	Variant	Median (ms)	P95 (ms)	P99 (ms)	Shots/s
Fresh	64	Baseline	4.519	7.090	7.826	1218
Fresh	64	Pruned	4.458	4.949	5.278	1753
Fresh	64	Mixed	4.647	5.259	5.431	1709
Fresh	128	Baseline	4.404	4.853	5.127	1600
Fresh	128	Pruned	4.438	4.930	5.168	1813
Fresh	128	Mixed	4.625	5.259	5.436	1715
Fixed	64	Baseline	4.461	4.907	5.169	1586
Fixed	64	Pruned	4.403	4.809	5.116	1821
Fixed	64	Mixed	4.713	5.369	5.547	1685
Fixed	128	Baseline	4.399	4.831	5.092	1599
Fixed	128	Pruned	4.440	4.848	5.132	1806
Fixed	128	Mixed	4.649	5.260	5.437	1710

Pruning changes median latency by approximately -1.85% to $+0.02\%$ at five rounds, -1.36% to $+0.93\%$ at nine rounds, and -1.68% to $+1.33\%$ at 25 rounds. These small changes show how little of the parameter reduction appears in median execution time.

Table 6.14: Detector-to-prediction execution at 25 rounds on the H200 NVL with host inputs. Latency uses batch one and throughput uses batch eight. Batch denotes training batch size.

Schedule	Batch	Variant	Median (ms)	P95 (ms)	P99 (ms)	Shots/s
Fresh	64	Baseline	4.638	5.039	5.299	833
Fresh	64	Pruned	4.560	4.930	5.196	1448
Fresh	64	Mixed	4.688	5.361	5.523	1383
Fresh	128	Baseline	4.596	5.010	5.257	836
Fresh	128	Pruned	4.559	4.932	5.176	1450
Fresh	128	Mixed	4.832	5.522	5.684	1366
Fixed	64	Baseline	4.501	4.888	5.137	840
Fixed	64	Pruned	4.561	4.930	5.169	1448
Fixed	64	Mixed	4.711	5.358	5.533	1380
Fixed	128	Baseline	4.553	5.005	5.238	838
Fixed	128	Pruned	4.555	4.950	5.196	1455
Fixed	128	Mixed	4.701	5.360	5.543	1387

Graph operations, intermediate tensors, dispatch, and synchronization remain part of the measured path.

Mixed precision raises median latency relative to the pruned parent in every condition: by 5.9–10.0% at five rounds, 4.2–7.0% at nine rounds, and 2.8–6.0% at 25 rounds. Extra casts and the retained single-precision graph aggregation are possible contributors. Operator-level profiling would help identify the dominant costs.

The nine-round fresh-data, batch-64 baseline has the largest tail latency: P95 is 7.09 ms and P99 is 7.83 ms. Its throughput is about 1,218 shots/s. This condition has a larger tail than the other nine-round baselines, whose P95 values are approximately 4.83–4.91 ms. The aggregate timings leave the cause of this variation unresolved.

For the fixed-data, batch-64 group, five-round throughput is approximately 1,902 shots/s for the baseline, 1,942 after pruning, and 1,798 with mixed precision. At nine rounds, the corresponding rates are 1,586, 1,821, and 1,685 shots/s. At 25 rounds, they are about 840, 1,448, and 1,380 shots/s. The 25-round pruned model therefore processes about 72.5% more shots per second than its baseline in this condition. These rates describe batches of eight. Similar batch-one medians can therefore accompany differences in batched processing capacity.

The larger nine- and 25-round graphs have nearly the same median single-shot latency as the five-round graphs under this measurement procedure. Together with the

throughput differences, this suggests that graph size interacts with batching and fixed execution overhead. All three studies record the same GPU and software versions, with one timing campaign per model. Repeated campaigns would help estimate timing variability.

6.4.3 Runtime memory

Table 6.15 compares fixed-data, batch-64 variants. The reported TensorFlow allocator peak for the baseline rises from 16.74 MiB at five rounds to 34.34 MiB at nine rounds and 70.22 MiB at 25 rounds. The larger graph therefore increases measured memory even though parameter count stays fixed.

Table 6.15: Detector-to-prediction memory for fixed-data, batch-64 models. Values are the maximum recorded peaks across samples. RSS uses 5 ms sampling.

Rounds	Variant	Allocator peak (MiB)	RSS peak (MiB)
5	Baseline	16.74	1308.4
5	Pruned	17.73	1294.1
5	Mixed	13.96	1307.7
9	Baseline	34.34	1386.9
9	Pruned	29.07	1413.6
9	Mixed	29.43	1400.0
25	Baseline	70.22	1720.1
25	Pruned	73.27	1670.7
25	Mixed	74.70	1641.3

Pruning slightly raises the five-round allocator peak and lowers the nine-round peak. Mixed precision lowers the five-round peak, while its nine-round value is close to that of the pruned model. At 25 rounds, the pruned and mixed variants reach 73.27 and 74.70 MiB, respectively, both above the baseline. Process RSS includes the TensorFlow runtime, graph collection, loaded models, and buffers. Its changes are small relative to the overall footprint and depend on the condition. Parameter storage gives the clearest compression benefit in these experiments.

All timing results concern complete-shot software processing. QPU measurements, controller communication, and Pauli-frame-dependent operations fall outside this scope. Evaluating the proposed system in a controller requires those components and a measured reference path.

6.5 Comparison across round counts

The 25-round study is complete, with test evaluations and timing measurements for all twelve variants under the frozen configuration.

The three studies share learning rate 10^{-4}, shot limits, training schedules, batch sizes, and gate and timing rules. Each baseline starts from fresh initialization. The five-round models learn predictions beyond the class imbalance, while the nine- and 25-round models have AUCs close to 0.5. Early stopping gives these studies different achieved training budgets, and the experimental records also change with round count. The comparison therefore describes the performance of this configuration on the three histories. Isolating the effect of history length would require controlled training budgets and additional initializations.

The larger 25-round graphs increase baseline allocator memory and reduce batch-eight throughput, while median single-shot latency remains close to the shorter-round measurements. Pruning improves throughput on these larger graphs without producing a comparable reduction in single-shot latency. The zero accepted coverage in all three studies remains the main obstacle to reducing reference-decoder work with the fitted gates.

The supplied reference-comparison report covers five and nine rounds, so the classical accuracy and threshold tradeoff at 25 rounds remain unmeasured in this analysis. The saved 25-round predictions allow the same paired comparison once the matching reference outputs are included.

7 Related work

Neural decoding research considers both replacing a classical decoder and assisting one with a learned first stage. We discuss examples of both approaches and explain how they relate to our use of a detector–error graph and a confidence gate.

7.1 Neural decoders

7.1.1 Early results

Torlai and Melko presented an early neural decoder for topological codes in 2017 [8]. They used a restricted Boltzmann machine and demonstrated the approach on a two-dimensional toric code with phase-flip errors. The model learns the joint statistics of error chains and their syndromes. During decoding, the observed syndrome constrains the sampling procedure, which searches for a compatible recovery chain.

This work demonstrates that a neural model can learn information useful for quantum error correction. Its output and execution procedure differ from ours: sampling produces a candidate recovery chain, whereas our feed-forward network estimates one logical-flip probability. We therefore use the paper as an early example of learned decoding, rather than as a directly comparable timing baseline.

7.1.2 Graph neural networks

Lange et al. formulate decoding as graph classification [9]. They construct annotated detector graphs from stabilizer measurements and use a graph neural network to predict the logical error class. Their approach learns from data without requiring a supplied noise model. They report performance beyond matching for simulated surface-code circuit noise and accuracy comparable to MWPM on experimental repetition-code data. The experimental result concerns repetition codes, so it does not by itself establish the same performance on recorded surface-code data.

Our approach also uses graph-level classification, but includes explicit error-mechanism

nodes and probabilities from the supplied DEM. We use this information as a prior instead of learning solely from measurement examples. We also ask a different deployment question: whether a small model can handle a reliable subset of shots before a reference decoder. Consequently, gate coverage and the cost of rejected shots matter alongside the neural model's overall prediction accuracy.

7.2 Neural predecoders

Chen et al. propose a two-stage neural residual decoder for surface codes [16]. Their preprint combines a GNN-based first stage with an MWPM decoder. The neural stage assigns confidence to local correction candidates and handles those that pass a threshold. The classical decoder then processes the remaining syndrome. Their study uses simulated circuit-noise data, so its results do not establish the performance of that pipeline on our experimental collection.

Both approaches use confidence to divide work between neural and classical components, but the division takes place at different levels. Chen et al. simplify a record before passing its residual syndrome to MWPM. Our gate instead accepts or rejects a complete logical prediction. A rejected shot reaches the reference decoder with its original detector events. This design avoids constructing a residual syndrome, but it can reduce reference-decoder invocations only when the entire neural prediction meets the acceptance criterion.

These studies motivate neural decoding and delegation, but their reported speed and error rates do not provide interchangeable baselines. They use different codes, data sources, outputs, and execution paths. We therefore compare our neural and reference predictions on shared shot identifiers and assess runtime within the measurement scopes defined in Chapter 6.

8 Summary and conclusion

We developed a neural predecoder that predicts a logical-observable flip from a full detector measurement history. The detector-error graph preserves the relationships and probabilities supplied by each sample's DEM. The model shares weights across samples, and a separate confidence gate decides whether a prediction can bypass the reference decoder. We implemented structured pruning, mixed-precision export, and a workflow that separates training, validation, calibration, and testing.

The completed experiments cover 5, 9, and 25 rounds of the distance-five, X-basis surface code. At five rounds, the fixed-data, batch-64 baseline reaches 62.31% accuracy and AUC 0.6908. Its pruned version reaches 62.32% accuracy. Both improve on the constant predictor's 55.27%. At 9 rounds, all 12 variants return the majority-class decision on every test shot, giving 51.45% accuracy and AUCs close to 0.5. The clearest improvement over the constant predictor occurs at 5 rounds. Correlated matching reaches 82.11% and 73.19% accuracy on the same test sets, outperforming every neural variant.

At 25 rounds, all twelve variants complete testing on 397,500 held-out shots. Accuracy ranges from 49.93% to 50.08%, and AUC ranges from 0.4994 to 0.5021. The fixed-data, batch-64 baseline improves on the training-selected constant predictor by 0.147 percentage points. Ten variants predict a logical flip on every shot. These results show little discrimination between the two labels on the longer histories. All twelve fitted gates have zero test coverage. The supplied classical-reference comparison remains limited to five and nine rounds.

Compression gives a clear storage benefit. Pruning reduces the parameter count from 75,266 to 28,290, and the mixed export stores 80.8% fewer parameter bytes than the baseline. Median single-shot latency changes little with pruning and increases with mixed precision. Across the completed variants, detector-to-prediction latency is approximately 4.39-4.83 ms on the H200 NVL. Pruning also improves batched throughput at 25 rounds. For the fixed-data, batch-64 condition, throughput rises from about 840 to 1,448 shots/s with inference batch size eight.

The research question asks whether a lightweight predecoder can reduce reference-

decoder work while preserving reliability and limiting its own cost. At the chosen 1% accepted-error target, every gate delegates every shot. The original fitted gates consequently save zero reference-decoder calls. In a serial implementation, its neural and gate execution would add to the reference path's cost.

The exploratory threshold comparison shows why neural confidence alone does not resolve this limitation. At 5 rounds, the fixed-data, batch-64 baseline accepts 1.284% of test shots at threshold 0.985. It reaches 98.37% accuracy on these shots, while correlated matching reaches 98.73% on the same subset. The hybrid corrects 15 reference errors and introduces 33, lowering overall accuracy by 0.00453 percentage points. Some thresholds accept very small subsets with no observed accuracy change, but the uncertainty bounds leave possible degradation unresolved. At 9 rounds, the configured sweep moves directly from accepting every shot at 0.5 to accepting none at 0.6 and above. The tested policies therefore provide no evidence of useful workload reduction with preserved overall accuracy. Further progress requires reliable accepted predictions relative to the classical decoder and enough saved work to cover the neural path's cost.

These findings concern one initialization seed and unseen shots from samples represented in training. The DEMs remain fixed, and the reports contain no live reference-decoder measurement.

9 Future work

The completed 25-round study supplies predictions and timing measurements for all twelve variants. Applying the paired reference comparison to these saved predictions would extend the accuracy–coverage results to longer detector histories. This requires the matching stored classical predictions and can reuse the completed neural evaluations.

The nine- and 25-round results motivate a closer study of optimization and graph representation. Experiments with several initialization seeds, matched training budgets, and controlled changes to the temporal representation could help explain the nearly constant predictions. Small-data memorization and held-out generalization checks would help distinguish the ability to fit records from learning a useful decoding rule.

Improving reliable accepted coverage is the next requirement for the delegation system. Future candidates should use training and validation data for model development and reserve independent calibration data for gate selection. A fresh held-out test set would support a final comparison after these changes. Per-sample analysis could also show whether a gate fitted to the pooled data provides useful coverage throughout the collection.

The threshold sweep motivates a selection rule that directly measures accuracy relative to the reference decoder. We could select a threshold on independent calibration data using paired error differences and a predefined tolerance for degradation, then assess it on new test shots. A denser threshold grid between 0.5 and 0.6 would also clarify the abrupt coverage transition observed at nine rounds. The present exploratory test results provide context for these choices, confirming a revised policy requires independent evidence.

The stored reference predictions establish the accuracy comparison. Running correlated matching on the benchmark machine would provide its latency, including its cost on the shots that the predecoder accepts. Timing the complete serial hybrid path would then show whether avoided reference calls compensate for neural inference and gating. The fraction of calls avoided alone leaves this execution-time benefit unresolved.

Finally, operator-level profiling could identify why fewer parameters produce only small

latency changes and why mixed precision slows this implementation. The measured graph aggregation, casting, and dispatch costs would guide further implementation work. Connecting the resulting system to a quantum controller would additionally require a Pauli-frame interface, streaming inputs, and measurements of the full control path.

Abbreviations

List of Figures

2.1 Schematic stabilizer supports of the surface-code patches. Red circles denote data qubits and blue circles denote ancilla qubits. The distance-five patch contains 25 data qubits and 24 ancilla qubits, the distance-three patch contains 9 data qubits and 8 ancilla qubits. Dashed lines indicate stabilizer support connections. 6

3.1 Conceptual quantum error-correction pipeline with speculative predecoding. The implemented prototype covers neural prediction, confidence gating, and inference benchmarking. Stored classical-decoder predictions support the offline quality comparison. The QPU, controller, and live reference path illustrate the intended application context. 12

4.1 Construction of the detector-error graph from the quantum circuit and detector error model. The circuit branch provides detector geometry and temporal information, while the detector error model provides candidate error mechanisms with their probabilities p_j, detector support $\mathcal{D}_j$, and logical support $\mathcal{L}_j$, where j is the error instruction index. 16

6.1 Training and validation BCE for the four 5-round baseline conditions. Each panel uses its own axis ranges. 36
6.2 Training and validation BCE for the four 9-round baseline conditions. Each panel uses its own axis ranges. 37
6.3 Training and validation BCE for the four 25-round baseline conditions. Each panel uses its own axis ranges. 38
6.4 Confidence-threshold comparison for the five-round, fixed-data, batch-64 baseline. The panels show overall accuracy, the accuracy of both decoders on the accepted subset, and the fraction of reference calls avoided. Thresholds occupy equally spaced positions for readability. Accepted-subset accuracy is undefined at threshold 1 because no test shots qualify. Table 6.10 resolves the small accuracy differences at high thresholds. 42

List of Tables

6.1 Completed round-count comparisons. Each variant has test and timing results. 28
6.2 Graph dimensions in the completed studies. Directed edges include self-loops. 29
6.3 Shared experimental settings. Epoch limits apply before early stopping. 30
6.4 Prediction quality at 5 rounds on 397,500 held-out shots. The constant predictor reaches 55.27% accuracy. Batch denotes training batch size. . . 33
6.5 Prediction quality at 9 rounds on 397,500 held-out shots. The constant predictor reaches 51.45% accuracy. Batch denotes training batch size. . . 34
6.6 Prediction quality at 25 rounds on 397,500 held-out shots. The constant predictor reaches 49.93% accuracy. Batch denotes training batch size. . . 34
6.7 Correlated matching and the highest neural test accuracy for each round count. Both use the same shots within each study. 35
6.8 Baseline training exposure and stage duration, including loading and validation. Presentations count repeated uses of a shot and are given in millions. 36
6.9 Frozen confidence gates at the 1% accepted-error target. The coverage column gives the range across models within each round count. Accepted-error ranges use nonempty accepted sets. 39
6.10 Fixed-threshold test comparison at 5 rounds for the fixed-data, batch-64 baseline. Coverage equals the fraction of reference calls avoided. 41
6.11 Parameter count and storage, common to the completed round counts. Payload counts exported arrays; resident storage counts loaded variables. 44
6.12 Detector-to-prediction execution at 5 rounds on the H200 NVL with host inputs. Latency uses batch one; throughput uses batch eight. Batch denotes training batch size. 45
6.13 Detector-to-prediction execution at 9 rounds on the H200 NVL with host inputs. Latency uses batch one; throughput uses batch eight. Batch denotes training batch size. 45

6.14 Detector-to-prediction execution at 25 rounds on the H200 NVL with host inputs. Latency uses batch one and throughput uses batch eight. Batch denotes training batch size. 46
6.15 Detector-to-prediction memory for fixed-data, batch-64 models. Values are the maximum recorded peaks across samples. RSS uses 5 ms sampling. 47

Bibliography

[1] Google Quantum AI and Collaborators. "Quantum error correction below the surface code threshold." In: *Nature* 638 (2025), pp. 920–926. DOI: 10.1038/s41586-024-08449-y.

[2] Google Quantum AI. "Suppressing quantum errors by scaling a surface code logical qubit." In: *Nature* 614.7949 (2023), pp. 676–681. DOI: 10.1038/s41586-022-05434-1.

[3] V. Sivak, M. Newman, and P. Klimov. "Optimization of Decoder Priors for Accurate Quantum Error Correction." In: *Physical Review Letters* 133 (2024), p. 150603. DOI: 10.1103/PhysRevLett.133.150603.

[4] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary. Cambridge University Press, 2010. ISBN: 9781107002173. DOI: 10.1017/CBO9780511976667.

[5] D. Gottesman. "Stabilizer Codes and Quantum Error Correction." PhD thesis. California Institute of Technology, 1997. arXiv: quant-ph/9705052.

[6] C. Gidney. "Stim: a fast stabilizer circuit simulator." In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497.

[7] A. G. Fowler. *Optimal complexity correction of correlated errors in the surface code*. 2013. DOI: 10.48550/arXiv.1310.0863. arXiv: 1310.0863 [quant-ph].

[8] G. Torlai and R. G. Melko. "Neural Decoder for Topological Codes." In: *Physical Review Letters* 119.3 (2017), p. 030501. DOI: 10.1103/PhysRevLett.119.030501.

[9] M. Lange, P. Havström, B. Srivastava, I. Bengtsson, V. Bergentall, K. Hammar, O. Heuts, E. van Nieuwenburg, and M. Granath. "Data-driven decoding of quantum error correcting codes using graph neural networks." In: *Physical Review Research* 7.2 (2025), p. 023181. DOI: 10.1103/PhysRevResearch.7.023181.

[10] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. "Mixed Precision Training." In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.03740.

[11] D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization." In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6980 [cs.LG].

[12] *Stim Result Formats: The b8 Format*. Stim contributors. URL: https://github.com/quantumlib/Stim/blob/main/doc/result_formats.md#the-b8-format (visited on 10/02/2026).

[13] Keras. *Mixed precision policy API*. URL: https://keras.io/api/mixed_precision/policy/ (visited on 09/27/2026).

[14] NIST/SEMATECH. *Bonferroni's Method*. e-Handbook of Statistical Methods, Section 7.4.7.3. Accessed 4 October 2026.

[15] C. J. Clopper and E. S. Pearson. "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial." In: *Biometrika* 26.4 (1934), pp. 404–413. DOI: 10.1093/biomet/26.4.404.

[16] J. Chen, J. Peng, and J. Liu. *Two-Stage Neural Residual Decoding for Fast Surface-Code Quantum Error Correction*. Preprint, version 1. 2026. DOI: 10.21203/rs.3.rs-10123128/v1.